

Bycatch Risk Assessment (ByRA) with **risa**

Joao Bosco Gusmao

2026-08-10

The **risa** package

The **risa** package, which stands for *RISk Assessment*, was designed to automate the application of the Habitat Risk Assessment (HRA) model, which is part of the *InVEST (Integrated Valuation of Ecosystem Services and Tradeoffs)* toolkit developed by the Natural Capital Project (Arkema et al., 2014; Sharp et al., 2020).

The package provides a suite of functions designed to automate the main steps required to apply the *ByRA* (Bycatch Risk Assessment) framework (Hines et al., 2020; Verutes et al., 2020). *ByRA* adapts the Habitat Risk Assessment (HRA) model to estimate bycatch risk by combining spatial overlap between fishing activities and marine megafauna with information on species vulnerability. By using the HRA model, risk is estimated from two main components: *exposure*, which describes the degree to which a species is likely to interact with a given fishery, and *consequence*, which represents the expected severity of that interaction for the species or population. Thus, *ByRA* integrates spatial patterns of fisheries and species occurrence with criteria related to susceptibility, sensitivity, and potential population-level impacts. Although *ByRA* has been applied mainly to assess bycatch risk for marine mammals, the HRA implementation provided in **risa** is flexible and can accommodate different formulations or extensions of the HRA model, following the general approach implemented in the *InVEST* toolkit.

When fitting the HRA model using **risa**, users can choose between multiplicative and Euclidean formulations of risk, apply optional decay functions to account for stressor influence buffers, and generate detailed summary statistics. In addition, **risa** provides flexibility to customize model settings, including the number of risk classes, buffer sizes for kernel density estimates and stressor zones, and the choice of coordinate system, either metric or decimal degrees.

Specifically, **risa** includes functions to generate kernel density maps of species occurrences and stressor intensities from either data frames or shapefiles, create overlap hotspot maps, and run the full HRA model for one or multiple species and stressors. It also includes functions for handling raster and vector maps, generating figures that summarize HRA results, exporting raster or vector outputs, and quickly implementing the *ByRA* framework from raw observation data, either for exploratory analysis or for formal applications.

Installation

The **risa** package requires R version ≥ 3.9 to work properly. However, using a more recent version of R is recommended. If your R installation is not up to date, you can download the latest version from:

<https://www.r-project.org/>

It is also recommended to install some of the core dependencies of **risa** beforehand. Although R usually installs package dependencies automatically, installing these core packages in advance can help avoid unexpected issues during the installation of **risa**.

```
# Some of the main dependencies
install.packages(c("sf", "terra", "dplyr", "tidyr"))

# And some packages we will use in this tutorial
```

```
install.packages(c("ggplot2", "readxl", "patchwork"))

# Loading
library(sf)
library(terra)
library(dplyr)
library(tidyr)
library(readxl)
library(ggplot2)
library(patchwork)
```

Since **risa** is still in its *alpha* version, it cannot be installed from the CRAN repository via `install.packages()`. Therefore, **risa** must be installed from its GitHub repository:

<https://github.com/GusmaoJB/risa/>

The easiest way to install **risa** is to use the `pak()` function from the **pak** package. If **pak** is not already installed, first install it with `install.packages("pak")`, and then run the code below to install **risa**.

```
# Load devtools
library(pak)

# install the risa package
pak::pak("GusmaoJB/risa")

# Load the risa package
library(risa)
```

Performing ByRA

First, let's keep our work files (dataframes) in a *data* folder inside our working directory.

You can set your working directory in RStudio by clicking in **Session**, then **Set Working Directory**, and finally **Choose Directory**. However, it is easier if you use `setwd()`.

```
# Replace the input with the path to your working project
setwd("/path/to/your/project/folder")
```

In this first example, let's suppose we have species and stressor distributions as occurrence points in CSV files. These are simulation data of observations of megafauna and fisheries in the Gulf of Guinea.

```
# Load important packages
library(sf)
library(terra)

# Loading species and stressor data
spp_df <- read.csv("data/species_input.csv")
str_df <- read.csv("data/stressor_input.csv")
```

Then we have a dataframe describing the occurrences of different species:

```
# Check the first rows
head(spp_df)
```

##	long	lat	species	group_size
## 1	-0.2961135	0.0008516	Dolphin	6
## 2	0.3154339	-0.2548119	Dolphin	1
## 3	-0.1913489	-0.0404221	Dolphin	2

```
## 4 -0.1840010  0.2328932 Dolphin      5
## 5 -0.3995284 -0.0046759 Dolphin      1
## 6 -0.0544592  0.1794313 Dolphin     16
```

And a data.frame with similar structure for stressors:

```
# Check the first rows
head(str_df)
```

```
##           long           lat stressor
## 1 -0.04256374 -0.25022293  Gillnet
## 2  0.08920605 -0.08777328  Gillnet
## 3 -0.15804650  0.03395657  Gillnet
## 4 -0.02735898 -0.12515094  Gillnet
## 5  0.20471062  0.32617057  Gillnet
## 6 -0.41094825 -0.30920867  Gillnet
```

All we need now is to load the criteria table. This table compiles the key information required for the analysis, including exposure ratings of each species (or habitat) to specific stressors, consequence ratings of those interactions, and the corresponding weights used in risk estimation and the respective quality score of the data.

Since we are working within the *ByRA* framework, we will use the criteria table specifically designed for this purpose, following the approach described in Hines et al. (2020) and Verutes et al. (2020).

```
# Load criteria data
criteria <- read.csv("data/criteria.csv")

# Check the first rows
head(criteria)
```

```
##           HABITAT.NAME Dolphin  X    X.1 Turtle X.2    X.3
## 1 HABITAT RESILIENCE ATTRIBUTES RATING DQ WEIGHT RATING DQ WEIGHT
## 2           age of maturity      3  3     2     3  3     2
## 3      reproductive strategy      3  3     3     1  3     3
## 4      population connectivity      2  3     2     1  3     2
## 5      local species status      2  3     2     3  3     2
## 6
##  CRITERIA.TYPE
## 1           E/C
## 2           C
## 3           C
## 4           C
## 5           C
## 6
##
## 1
## 2
## 3
## 4 (3) negligible movement/exchange between the focal regional population and other populations (a DP
## 5
## 6
```

Now we already have everything we need to run our first *ByRA*!

Let's start with a very simple example, where we just want a quick assessment of bycatch risk for two megafaunal species exposed to two different fishing gears. Since this is a first try, we will keep things straightforward and rely on the default behavior of the `quick_byra()` function.

This means we will not worry (yet) about the details behind how kernel density estimates for species and stressors are constructed, or how the HRA model is calculated. The default settings handle all of that for us, using conservative assumptions and standard data preparation protocols typically recommended for *ByRA* analyses.

We can always refine and tweak both the data prep and the HRA model later. So, for now, let's start simple and see what we get. We will add the argument *quiet=TRUE* to get some basic info about the construction of the mapped Kernel Density Estimates (KDE), and the default parameters for the HRA model estimates.

```
byral <- quick_byra(spp_df, str_df, criteria, quiet = FALSE)
```

```
## No area provided; creating AOI from stressor layers (convex_hull).
## Could not evaluate latitude span; using center latitude for polar/UTM decision.
## Auto-selected UTM zone 30: EPSG:32630.
## Auto pixel_size (square): 314.552 m
## Building KDE maps...
## Processing: Dolphin
## Auto bandwidth (nndist) : sigma = 7668 meters.
## Processing: Turtle
## Auto bandwidth (nndist) : sigma = 9131 meters.
## Processing: Gillnet
## Auto bandwidth (nndist) : sigma = 8325 meters.
## Processing: Trawling
## Auto bandwidth (nndist) : sigma = 6002 meters.
## Generating overlap maps...
## Estimating overlap map considering method: product
## Reclassifying overlap map based on 3 classes.
## Estimating overlap map considering method: product
## Reclassifying overlap map based on 3 classes.
## Estimating overlap map considering method: product
## Reclassifying overlap map based on 3 classes.
## Estimating overlap map considering method: product
## Reclassifying overlap map based on 3 classes.
## Risk equation: euclidean
## Max Rating score: 3
## Number of overlapping stressors: 2
```

And that is it!

The information printed in the Console when we set *quiet = FALSE* provides useful details about the workflow. First, the function automatically guessed the coordinate reference system of the input data as **UTM zone 30N: EPSG:32630**, which is appropriate for these data.

Then, during the KDE step, it automatically estimated the bandwidth used as the spatial radius (**sigma**, in meters) for each species and stressor. In this example, the bandwidth was estimated using the **nearest-neighbor distance** method (**nndist**).

The Console output also indicates that the overlap between species and stressor KDEs was calculated using the product of their reclassified KDE scores.

The resulting overlap values were then reclassified into three classes. Finally, the output shows that the HRA risk calculations were based on the Euclidean equation, with the maximum Rating value set to 3 and the number of overlapping stressors equals 2.

All results, including the maps and risk estimate statistics, are stored in *byra1*. If we inspect this object, we will find the following:

```
names(byra1)

## [1] "Dolphin"          "Turtle"           "ecosys_risk_raw"
## [4] "ecosys_risk_classified" "summary_stats"    "kde_maps"
## [7] "area_of_interest"
```

Which means:

- **Dolphin and Turtle:** Partial results (risk estimates) for each species (can be just one or more!). This is a list containing maps of the Exposure and Consequence components of the risk assessment, as well as overall risk estimates for each individual stressor and their combined effects. It also includes a table with summary statistics for these estimates.
- **ecosys_risk_raw:** A **SpatRaster** map of the raw ecosystem risk estimates, representing the combined risk across all species considered in the assessment (Dolphin and Turtle).
- **ecosys_risk_classified:** A **SpatRaster** map of reclassified ecosystem risk estimates. *ByRA* typically uses four risk categories: 0 = None, 1 = Low, 2 = Medium, and 3 = High.
- **summary_stats:** A **data.frame** with summary statistics of risk estimates at both the ecosystem and species levels. It includes results for individual and combined stressor effects, as well as minimum, maximum, and mean values of Exposure, Consequence, and risk estimates, in addition to the relative area of each risk category.
- **kde_maps:** A list containing all kernel density estimate maps for each species and stressor, as well as maps of overlap for each species–stressor pair.
- **area_of_interest:** An **sf** vector object representing the polygon that defines the area of interest for the *ByRA* analysis. If not provided by the user, it is estimated by default based on the spatial distribution of the stressors.

First, let's check the summary stats!

```
byra1$summary_stats
```

##	SPECIES	STRESSOR	E_min	E_max	E_mean	C_min	C_max	
## 3	ECOSYSTEM (FROM ALL STRESSORS)		NA	NA	NA	NA	NA	
## 1	Dolphin (FROM ALL STRESSORS)		0	2.470588	1.490708	0	2.529412	
## 2	Dolphin	Gillnet	0	2.470588	1.854817	0	2.529412	
## 31	Dolphin	Trawling	0	1.655172	1.126598	0	1.050000	
## 4	Turtle (FROM ALL STRESSORS)		0	2.647059	1.675527	0	2.750000	
## 5	Turtle	Gillnet	0	2.647059	1.864392	0	2.277778	
## 6	Turtle	Trawling	0	2.250000	1.486662	0	2.750000	
##	C_mean	R_min	R_max	R_mean	R.high	R.medium	R.low	R.None
## 3	NA	0	3.7597761	2.1963048	0.00000	62.12660	16.59444	21.27895
## 1	1.3080084	0	2.1217281	0.9523704	0.00000	68.62354	17.33893	14.03753
## 2	1.9075551	0	2.1217281	1.5101969	23.59318	57.77561	0.00000	18.63121

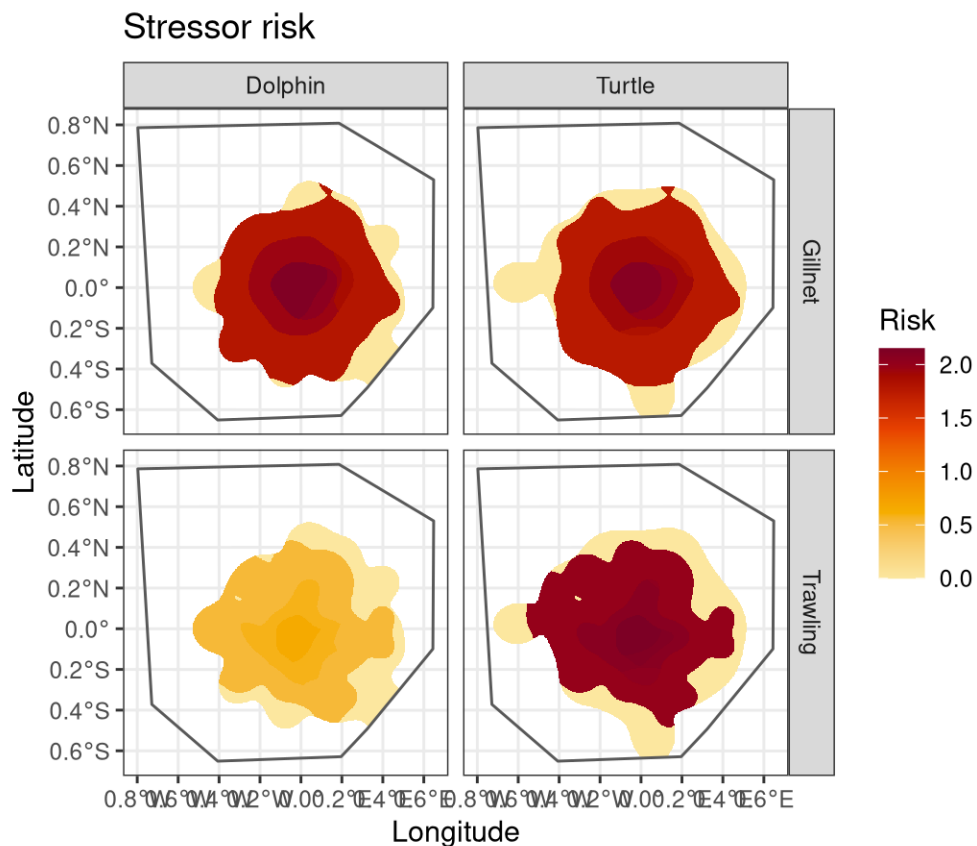
```
## 31 0.7084618      0 0.6570775 0.3945438  0.00000  0.00000 73.21722 26.78278
## 4  1.7233424      0 2.1505813 1.3938719 24.68082 44.69182 13.04506 17.58230
## 5  1.5967530      0 2.0845908 1.3857284 21.10897 54.88883  0.00000 24.00220
## 6  1.8499317      0 2.1505813 1.4020153 69.37264  0.00000  0.00000 30.62736
```

Now let's visualize the main results using the `risaplot()` function. Since we are working with multiple species and stressors, the function will generate six plots.

The first two plots show the risk estimates (raw and reclassified) for each species, considering the isolated effect of each stressor. Plots 3 and 4 display the risk estimates (raw and reclassified) for each species when all stressors are combined. Finally, plots 5 and 6 present the ecosystem-level risk estimates (raw and reclassified), derived from the combined risk across all species. Note: The ecosystem mode is absent when we have only one species.

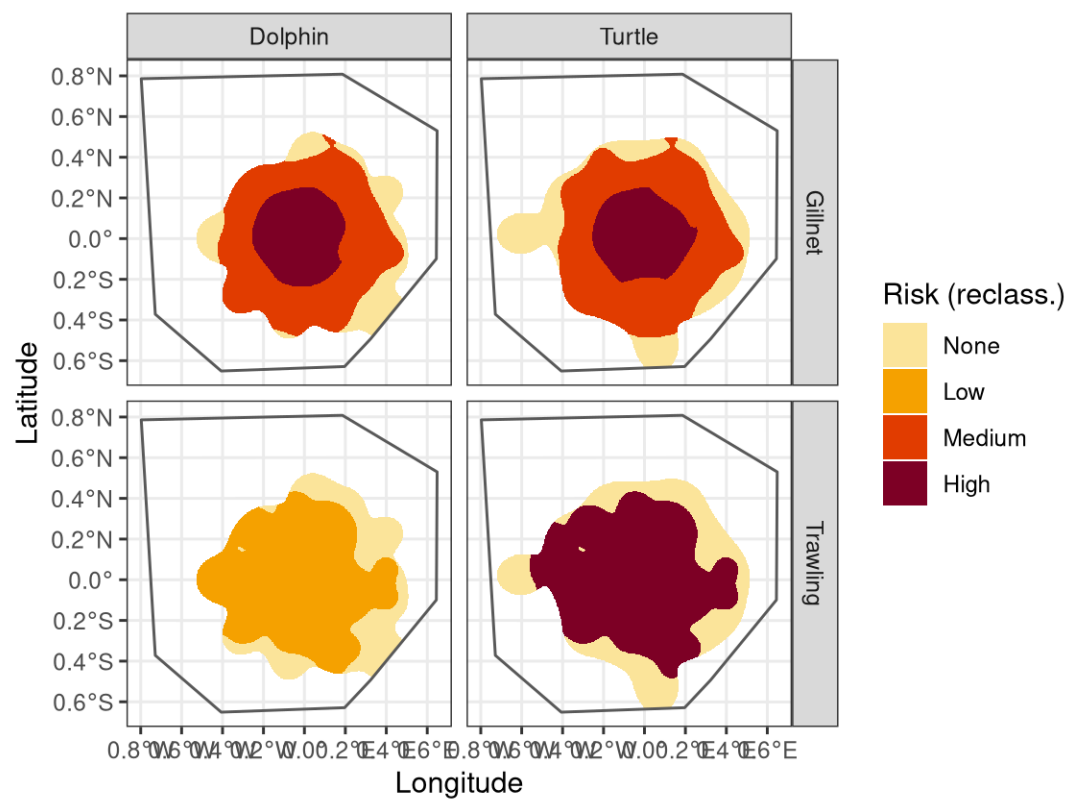
```
risaplot(byra1)
```

```
## [[1]]
```

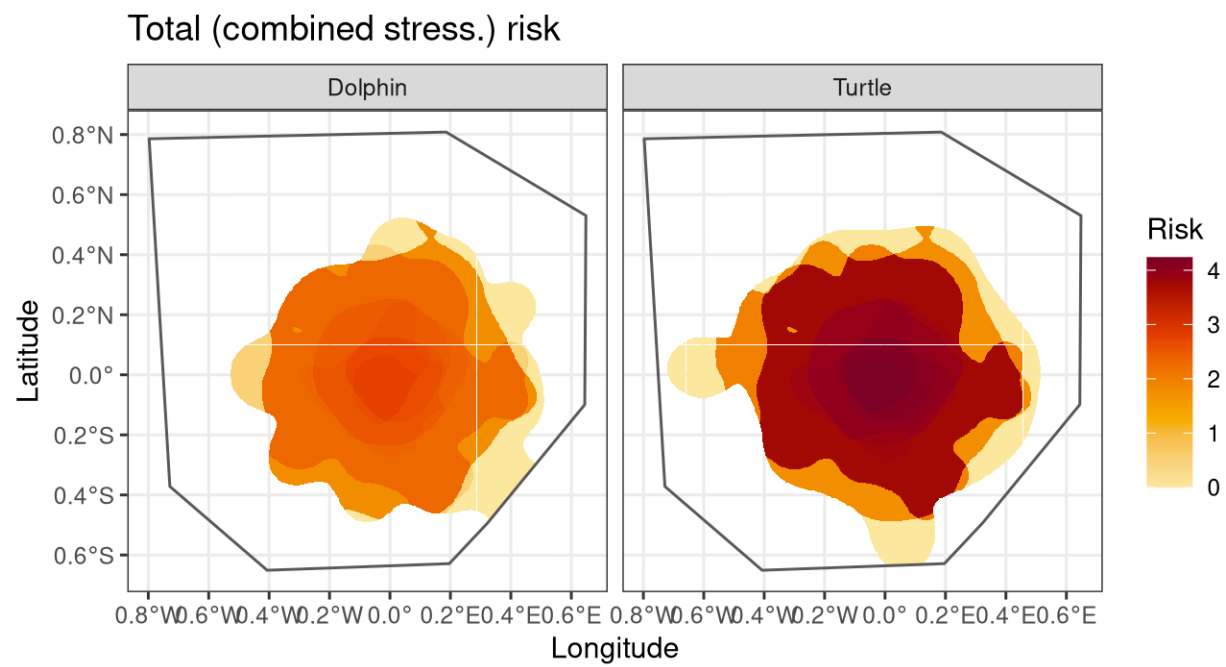


```
##
## [[2]]
```

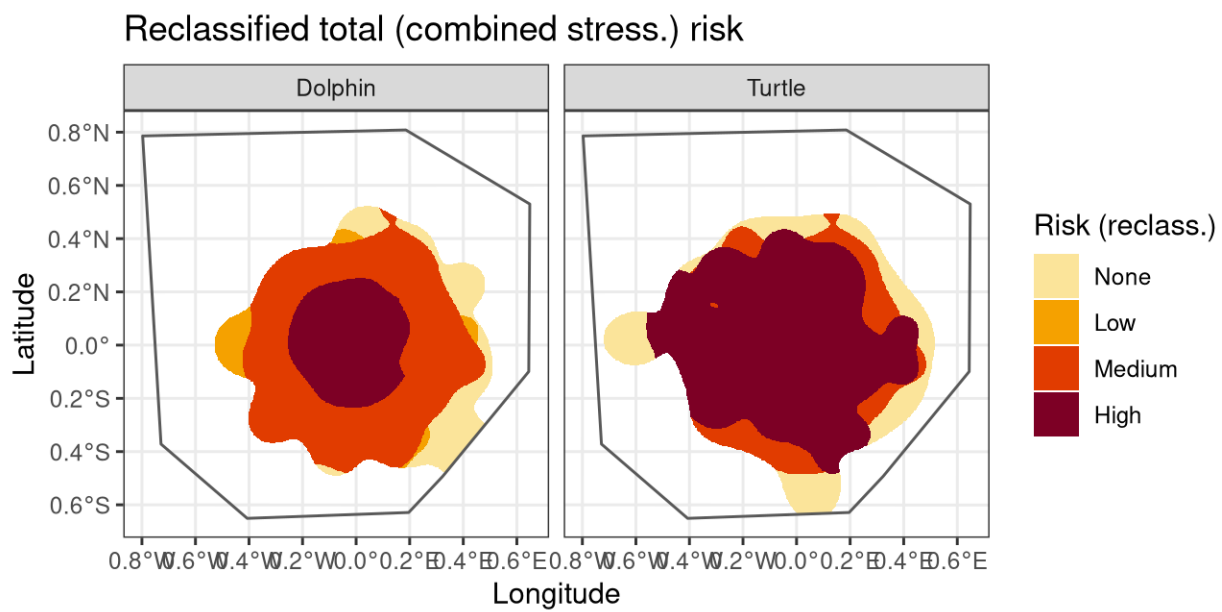
Reclassified stressor risk



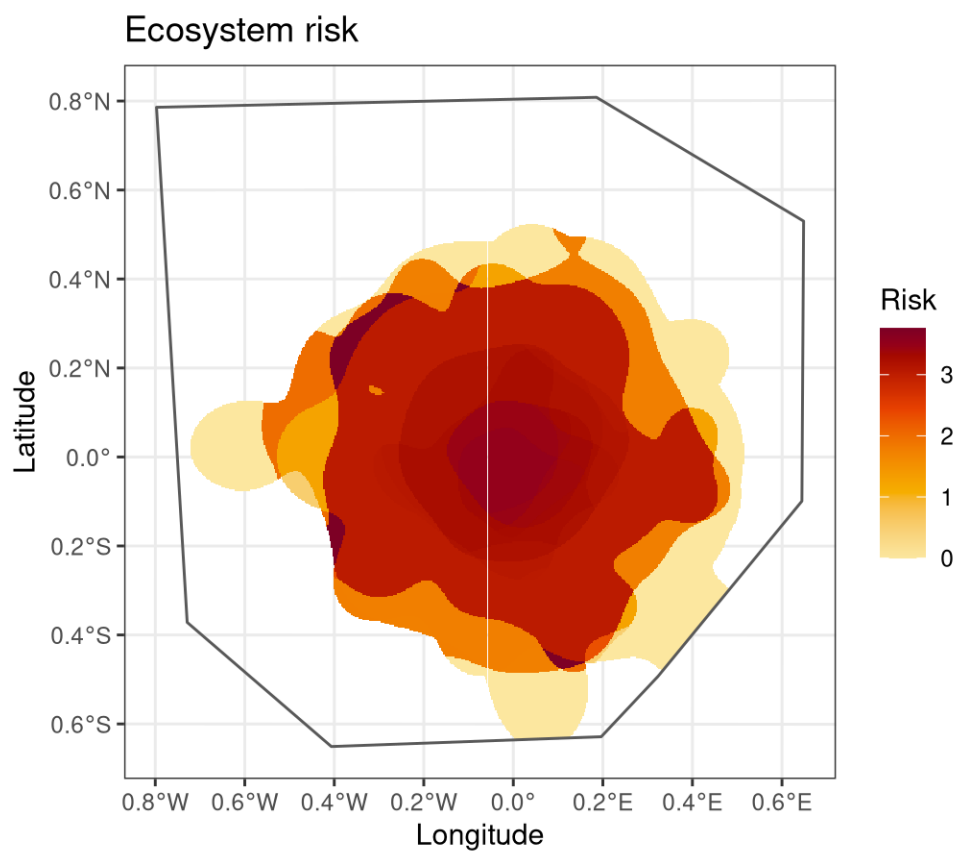
[[3]]



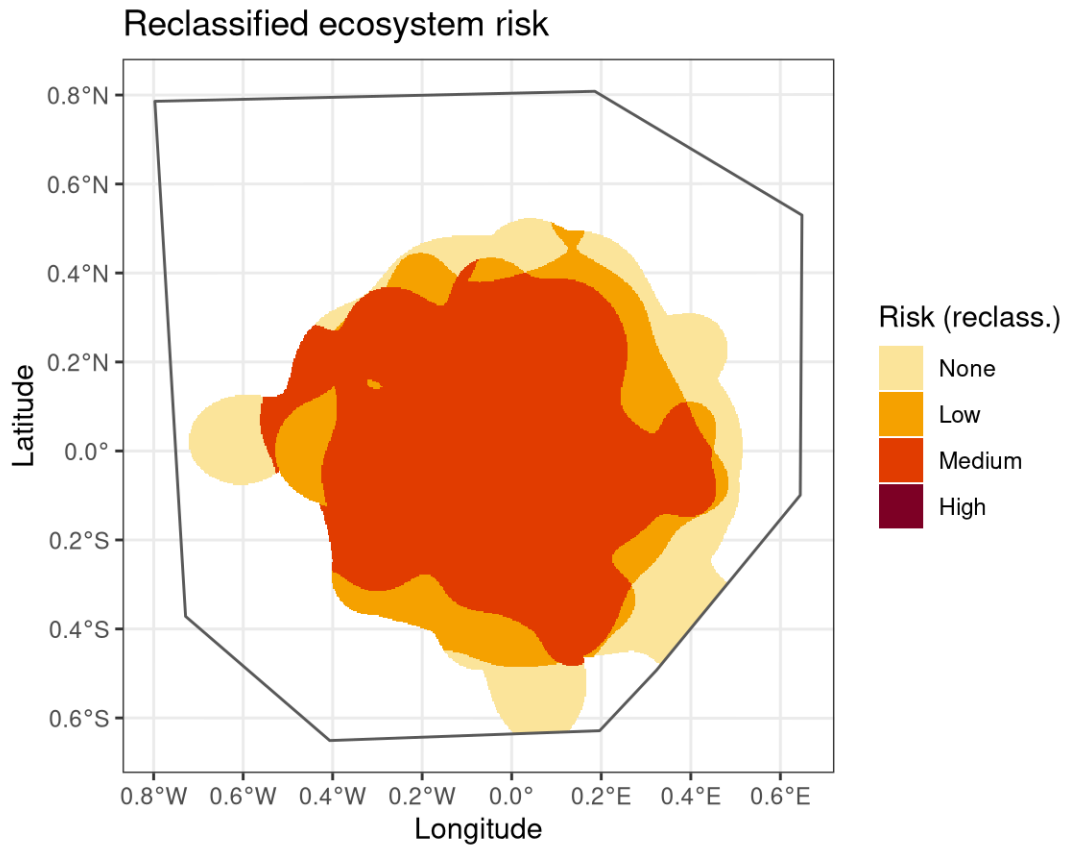
[[4]]



[[5]]



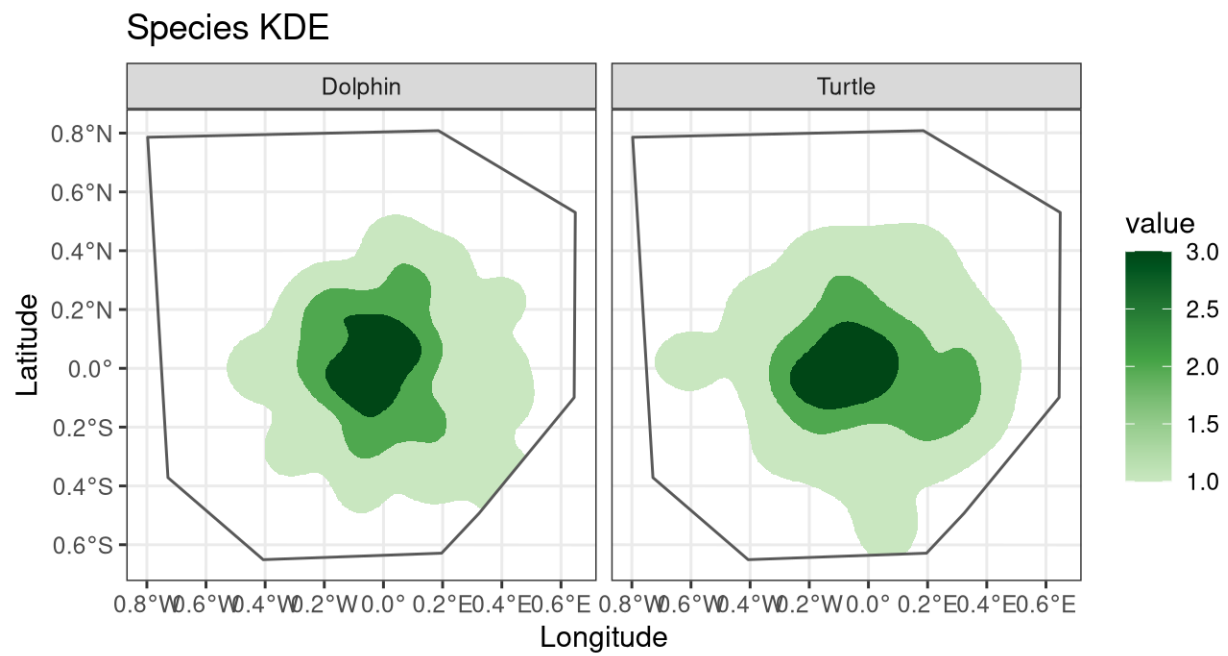
```
##  
## [[6]]
```



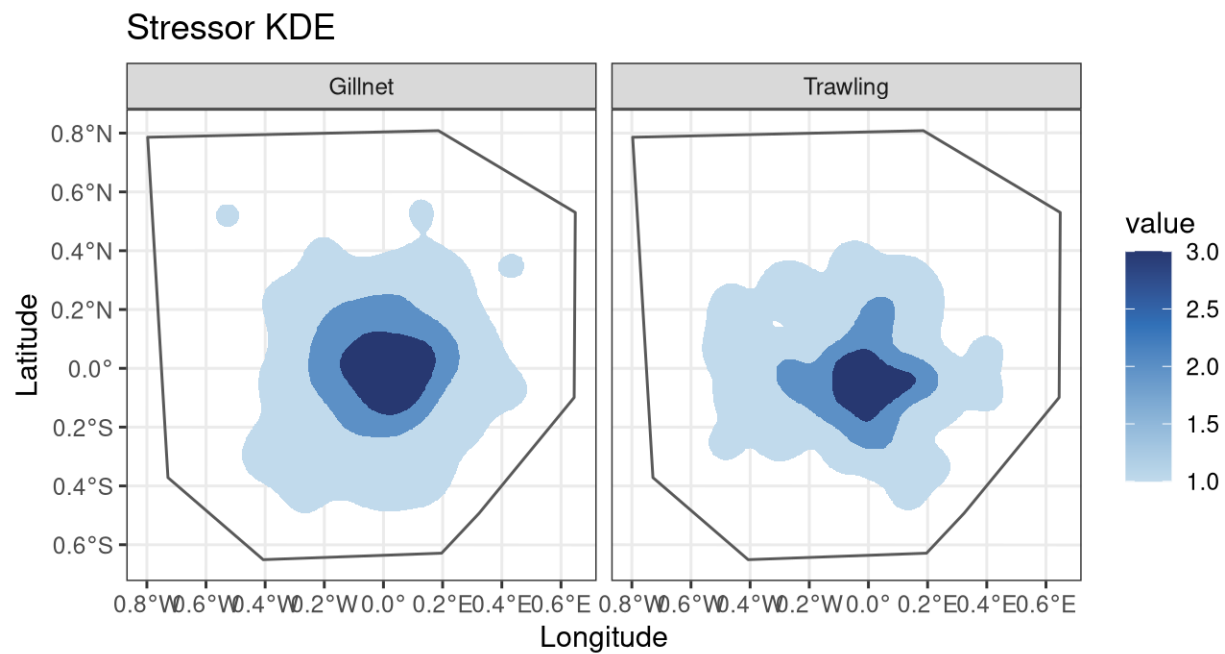
We can also visualize the maps that were considered for the risk estimates (KDE maps and their overlaps) using `risaplot()` applied to the `byra1$kde_maps` object. This will generate three plots: the first shows the KDE maps for each species, the second displays the KDE maps for each stressor, and the third presents the (reclassified) overlaps between each species–stressor pair.

```
risaplot(byra1$kde_maps)
```

```
## [[1]]
```

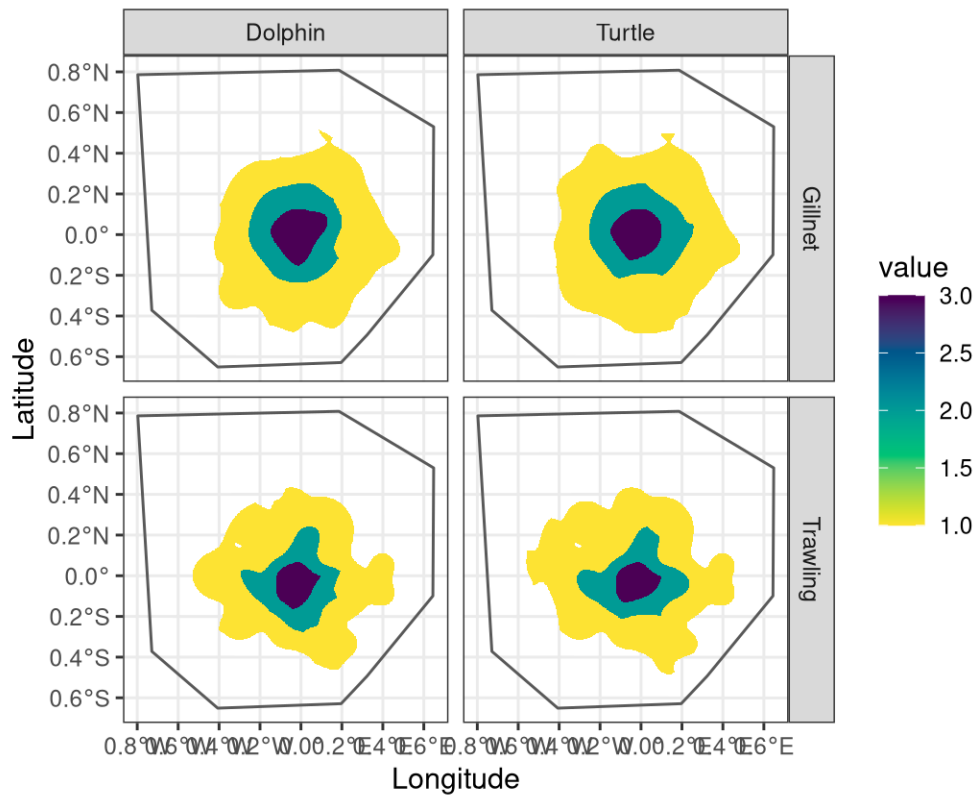


```
##  
## [[2]]
```



[[3]]

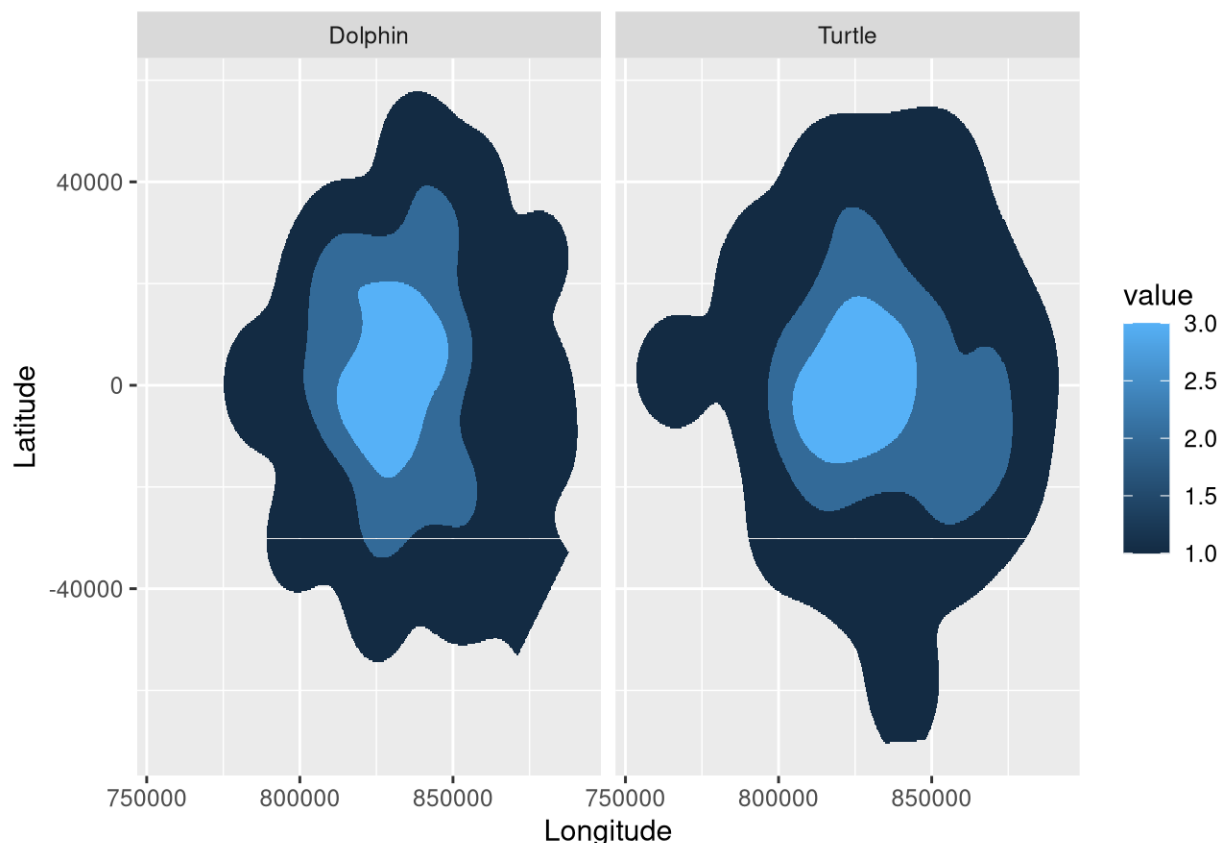
Species-Stressor Overlaps



If you would like to customize the default appearance of these plots, including colors, titles, labels, or other graphical elements, you can return a standard **ggplot2** object by setting `raw_ggplot = TRUE`. This allows the plot to be further modified using functions from the **ggplot2** package. In addition, the polygon representing the area of interest can be omitted by setting `area_of_interest = FALSE`.

```
default_plots <- risaplot(byra1$kde_maps,
  raw_ggplot = TRUE,
  area_of_interest = FALSE)

# Let's exemplify with the first plot
default_plots[[1]]
```



Calibrating ByRA

The example above provides a useful exploratory starting point. However, we can now refine the analysis by incorporating what we know about our data and study system.

First, we may want to work with a broader area of interest instead of restricting the analysis to the spatial distribution of the stressors. To do this, we can set: `area_strategy = "union"`, which uses the combined spatial extent of both species and stressor observations to define the area of interest.

Second, we can define different KDE bandwidths according to the expected distribution patterns of each species and stressor. Let's suppose we want to use a larger radius for **Dolphin** observations and a more restricted radius for **Turtle** observations, such as 12000 m and 7000 m, respectively. We can do the same for the stressors, and use a radius of 10000 m for **Gillnet** and 8000 m for **Trawling**.

Now, let's pass these values to `quick_byra()` using a named vector:

```
radius = c(Dolphin = 12000,
           Turtle = 7000,
           Gillnet = 10000,
           Trawling = 8000)
```

Another useful refinement concerns the KDE estimates for the species. Because we have information on the group size of each species observation, we can use this variable as a weight in the KDE calculations. This is done by providing the name of the column that stores group size information: `group_x = "group_size"`

Regarding the HRA model calculations, we are interested in the combined effects of multiple stressors. Therefore, we might want to use the **multiplicative** HRA equation, which is especially useful when estimating risk from multiple overlapping stressors.

Together, these arguments allow the *ByRA* workflow to better reflect the ecological characteristics of the species, the spatial distribution of the stressors, and the structure of the available data.

```
byra2 <- quick_byra(spp_df, str_df, criteria,
  area_strategy = "union",
  radius = c(Dolphin = 12000,
    Turtle = 7000,
    Gillnet = 10000,
    Trawling = 8000),
  group_size_x = "group_size",
  equation = "multiplicative",
  quiet = FALSE)

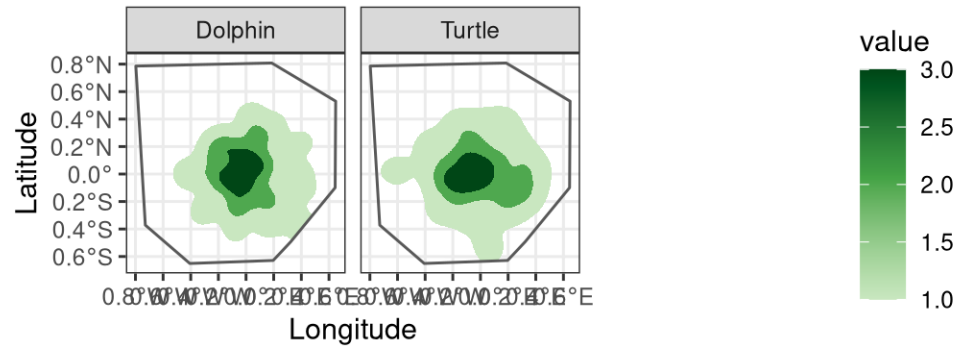
## No area provided; creating AOI from union layers (convex_hull).
## Could not evaluate latitude span; using center latitude for polar/UTM decision.
## Auto-selected UTM zone 30: EPSG:32630.
## Auto pixel_size (square): 336.881 m
## Buiding KDE maps...
## Multiple radius values provided for KDE.
## Processing: Dolphin
## Processing: Turtle
## Processing: Gillnet
## Processing: Trawling
## Generating overlap maps...
## Estimating overlap map considering method: product
## Reclassifying overlap map based on 3 classes.
## Estimating overlap map considering method: product
## Reclassifying overlap map based on 3 classes.
## Estimating overlap map considering method: product
## Reclassifying overlap map based on 3 classes.
## Estimating overlap map considering method: product
## Reclassifying overlap map based on 3 classes.
## Risk equation: multiplicative
## Max Rating score: 3
## Number of overlapping stressors: 2
```

Let's compare the KDE maps to see the differences in the input maps! This time, let's use an useful package called *patchwork* to make multi-panel plots.

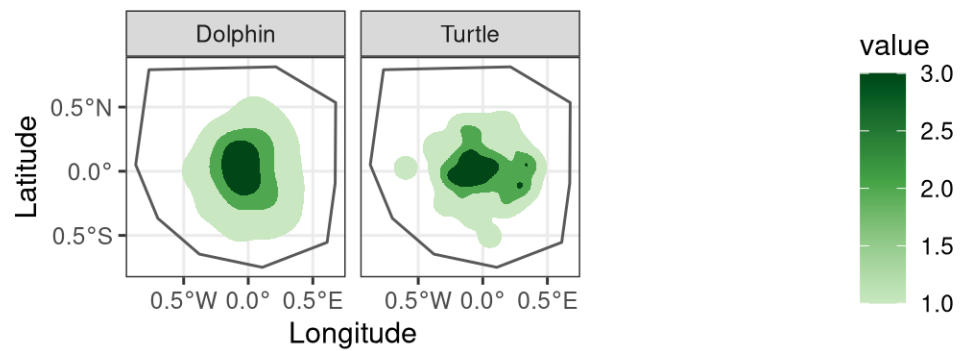
```
byra1_kdes <- risaplot(byra1$kde_maps)
byra2_kdes <- risaplot(byra2$kde_maps)

byra1_kdes[[1]] / byra2_kdes[[1]]
```

Species KDE



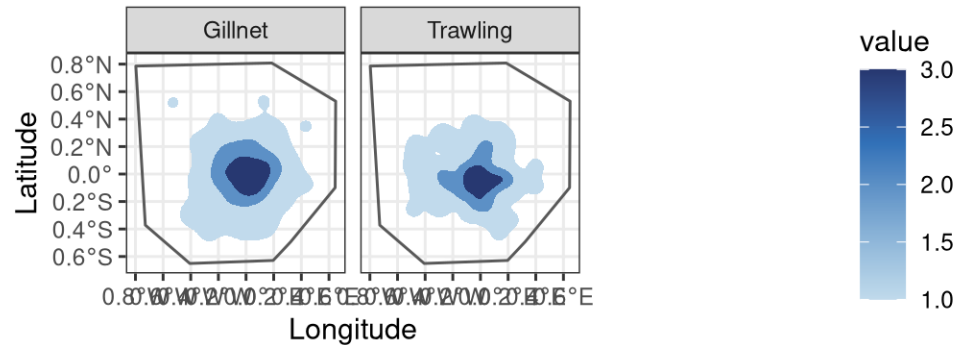
Species KDE



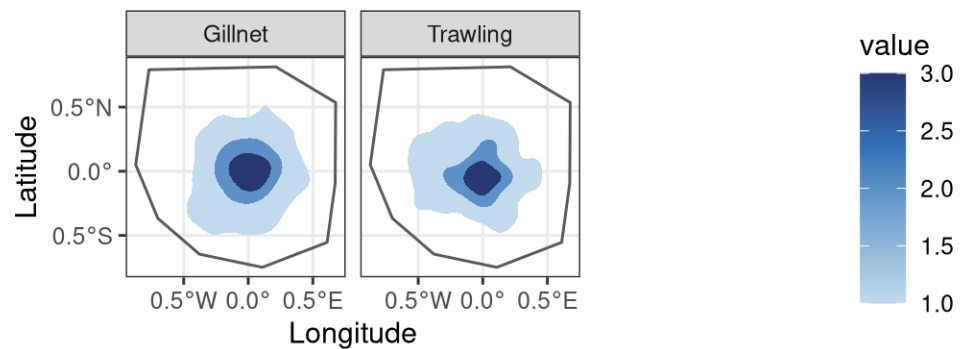
The output looks quite different! Now, let's check the stressors' KDEs!

```
byra1_kdes[[2]] / byra2_kdes[[2]]
```

Stressor KDE



Stressor KDE



As shown above, decisions made before the analysis can strongly influence the mapped distribution of species and stressors, as well as the resulting risk estimates. When applying *ByRA*, we should be particularly careful to avoid type II errors, such as assuming that a species does not occur in an area where it is actually present (Hines et al., 2020; Verutes et al., 2020).

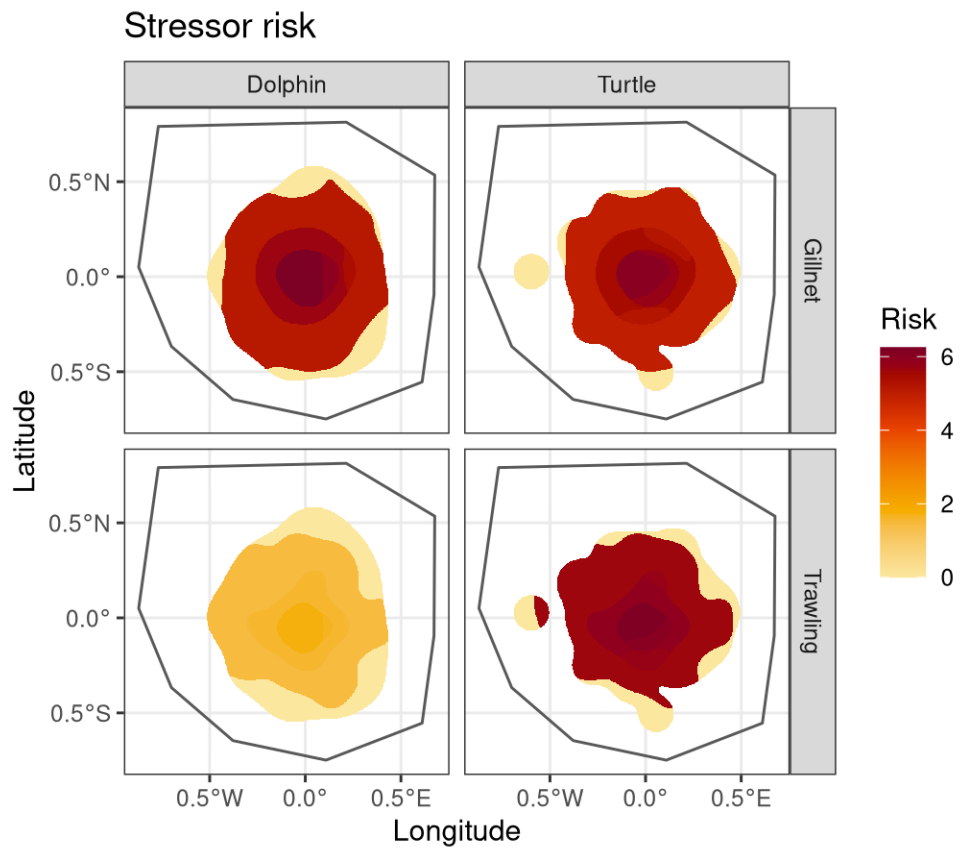
By default, **risa** estimates the radius used in KDE as the mean nearest-neighbour distance among points multiplied by 1.5. Thus, the radius is based only on the spatial dispersion of the observations. It does not account for the spatial scale of the analysis, nor for how that scale relates to the movement ecology of the species or to the spatial influence of the stressor being mapped.

The default radius estimates used to build KDE maps in **risa** may often provide reasonable representations of distribution hotspots. However, we strongly recommend treating KDE maps generated automatically from default settings as exploratory tools. Users should consider calibrating the KDE radius based on the **risa** outputs, the ecological characteristics of the species, and available information on its documented occurrence in the area of interest. The same rationale applies when building KDE maps to describe stressor distributions, for which the radius should also be evaluated in relation to the expected spatial extent or influence of the stressor.

Now, let's check how the mapped risk estimates look.

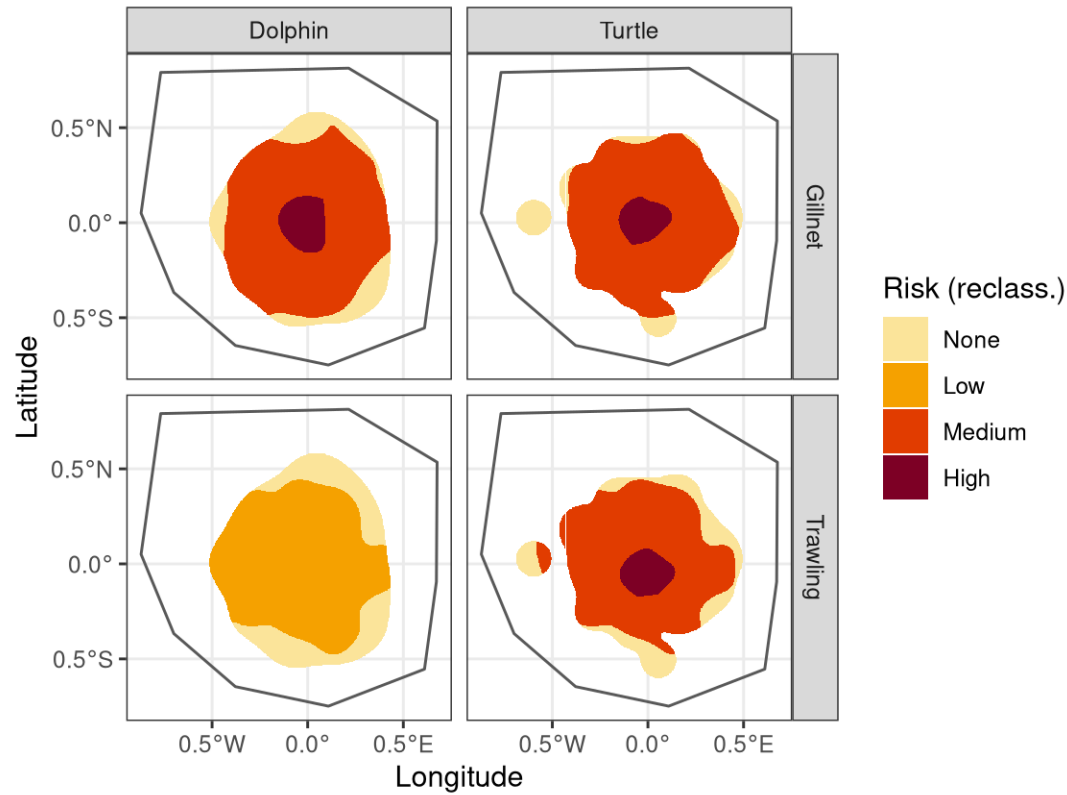
```
risaplot(byra2)
```

```
## [[1]]
```

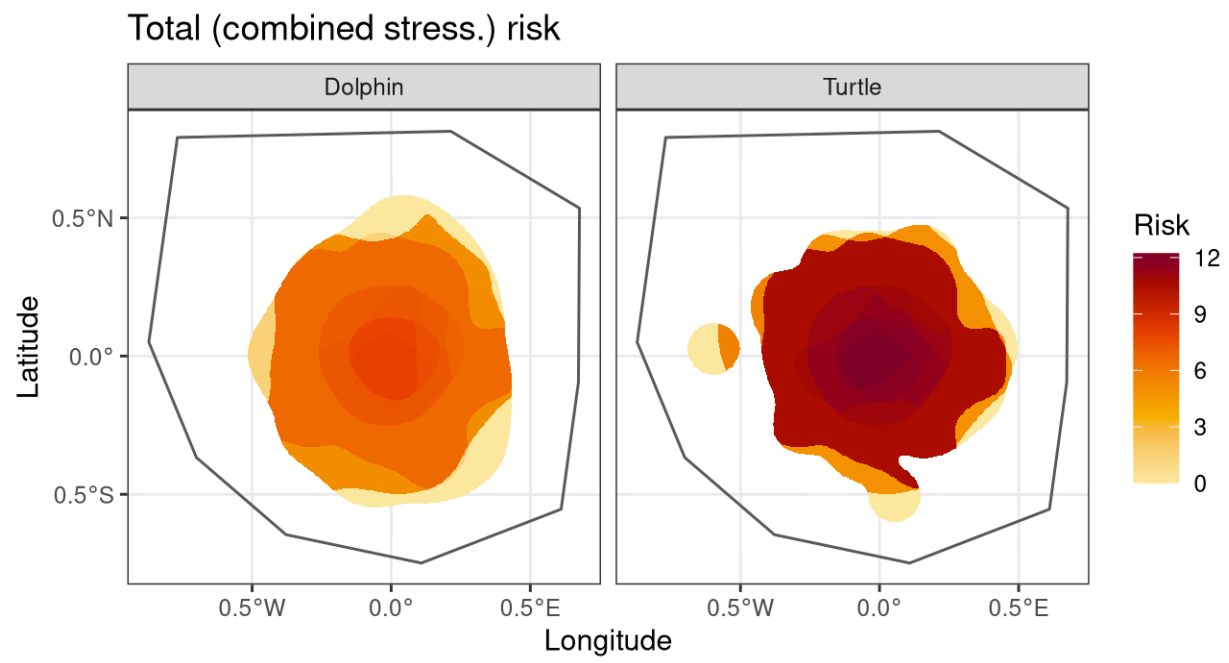


[[2]]

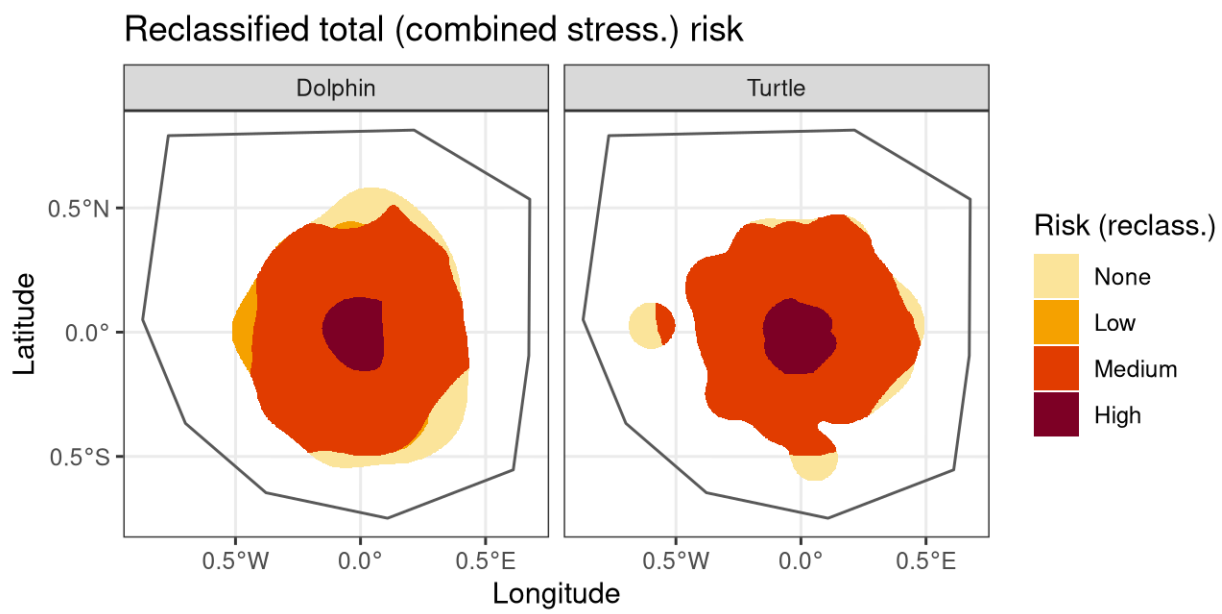
Reclassified stressor risk



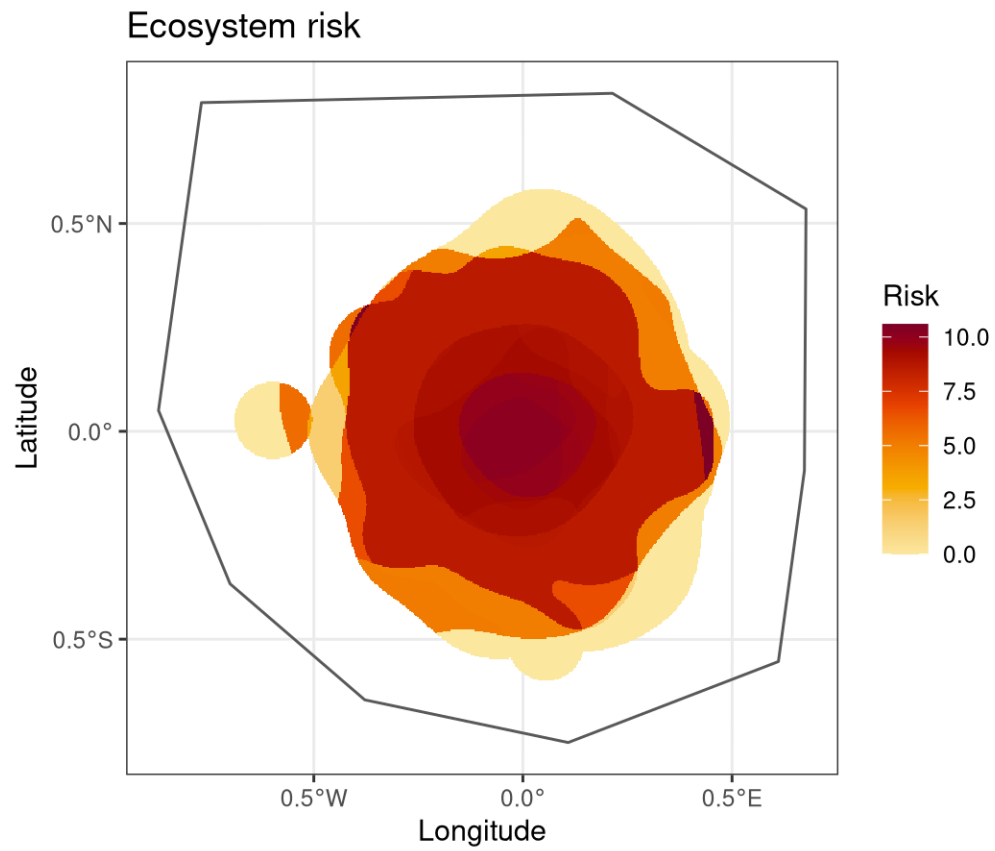
[[3]]



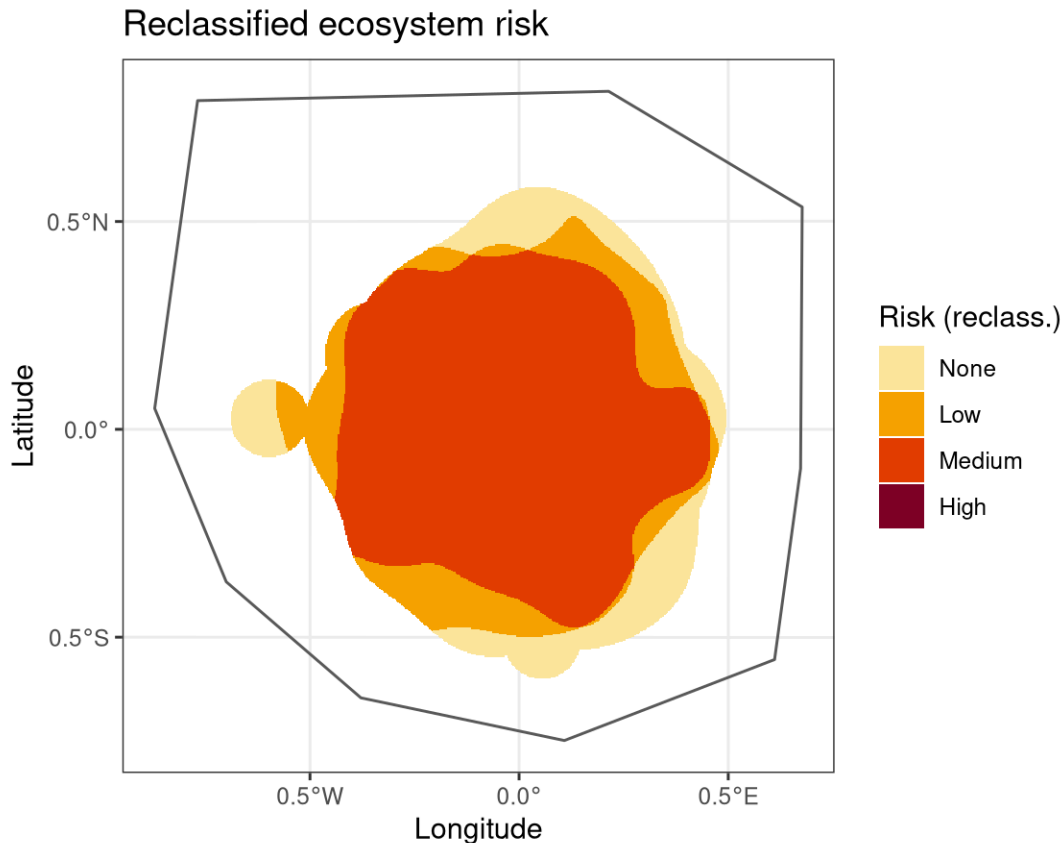
```
##  
## [[4]]
```



[[5]]



```
##  
## [[6]]
```



Running the model using layer inputs

In some cases, we may want to use ready-to-use spatial layers as input data for our analyses. These inputs can include vector files, such as shapefiles or *Google Earth KML/KMZ* files, and raster files, such as *GeoTIFF* or *ESRI ASCII Grid* files. We may also have spatial objects already loaded or created in *R*, such as species distribution models stored as **SpatRaster** objects from the **terra** package, or fishing areas represented as geospatial polygons stored as **sf** vector objects from the **sf** package.

In this example, we will use a hypothetical case with two species and two stressors. The data for the first species, *Seal*, are stored as a *GeoTIFF* file representing species distribution hotspots. The data for the second species, *Sea.lion*, are stored in an **Excel** spreadsheet containing georeferenced occurrence records. The first stressor, *Driftnet*, is stored as a collection of *Google Earth KML/KMZ* files in one folder (**driftnet_polygons**), while the second stressor, *Gillnet*, is stored as a shapefile in another folder (**gillnet_polygons**). All these files are stored inside a folder called **data2**.

We will load these files and harmonize their spatial structure so that they can be used as inputs to run *ByRA*.

First, let's load data about the two species, one of the stressors (gillnet) and the criteria table.

```
# Load Sea lion data from an Excel file
sea_lion_df <- readxl::read_excel("data2/sea_lion_df.xlsx")

# Load Seal data from a .tif raster
sea_rast <- terra::rast("data2/seal_hotspots.tif")

# Load Gillnet data from a shapefile
gillnet_sf <- sf::read_sf("data2/gillnet_polygons/gillnet_polygons.shp")
```

```
# Load criteria data
criteria2 <- read.csv("data2/criteria2.csv")
```

The data on driftnets is stored in multiple KML files, so it is a bit more complicated to load:

```
# First we need the path to the folder where we keep the kml files
folder_path <- paste(getwd(), "data2/driftnet_polygons", sep = "/")

# Then we recover the file names in that folder
kml_files <- list.files(
  folder_path,
  pattern = "\\\\.kml$",
  full.names = TRUE,
  ignore.case = TRUE
)

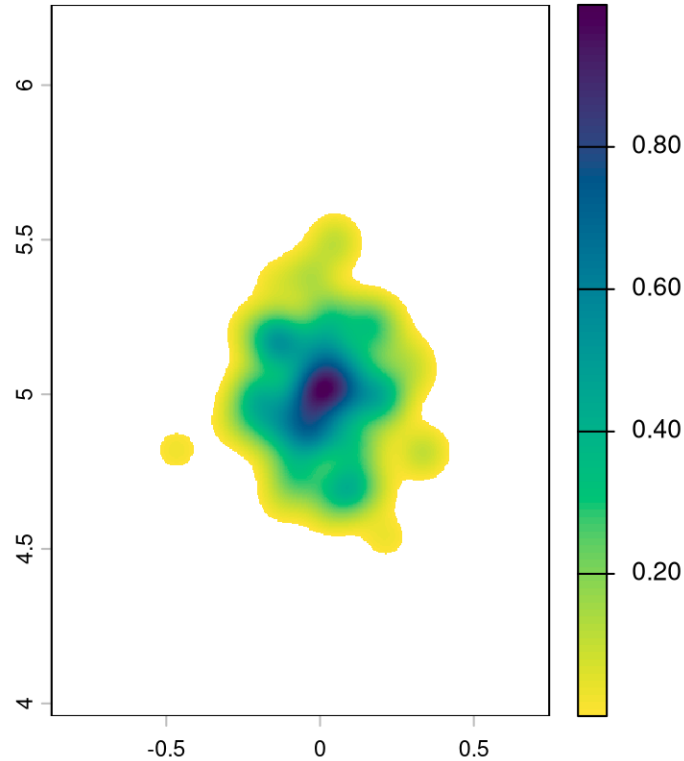
# Then we load all files as sf vectors and store them in a list
kml_list <- lapply(kml_files, function(f) {
  x <- sf::st_read(f, quiet = TRUE)
  x$source_file <- basename(f)
  return(x)
})

# Then we bind all sf vectors together into a single object
driftnet_sf <- dplyr::bind_rows(kml_list)
```

Now, we need to prepare the input maps.

The object `seal_rast` is almost ready to use. It is a raster layer with continuous values representing the probability of seal occurrence within a given area.

```
terra::plot(seal_rast, col=hcl.colors(100, rev=TRUE))
```

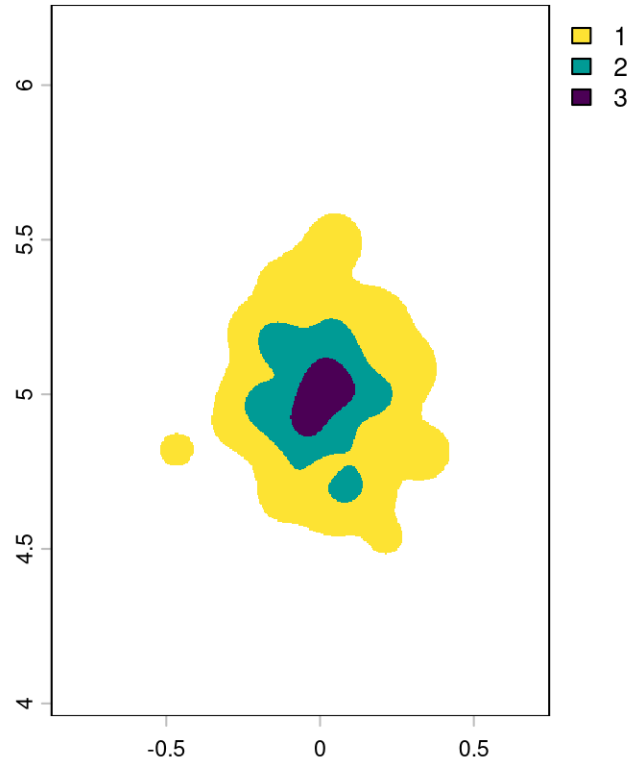


Although we could theoretically use these continuous values directly, the standard *ByRA* framework is usually based on ordinal integer categories, preferably ranging from 1 to 3. Therefore, we will reclassify the values in `seal_rast` into three categories.

```
# First we need to make a reclassification matrix
re_mat <- reclass_matrix(seal_rast,
                        n_classes = 3,
                        exclude_lowest = FALSE)

# Then use the reclassification matrix to reclassify our raster
seal_rast_rec <- terra::classify(
  seal_rast,
  re_mat,
  include_lowest = TRUE
)

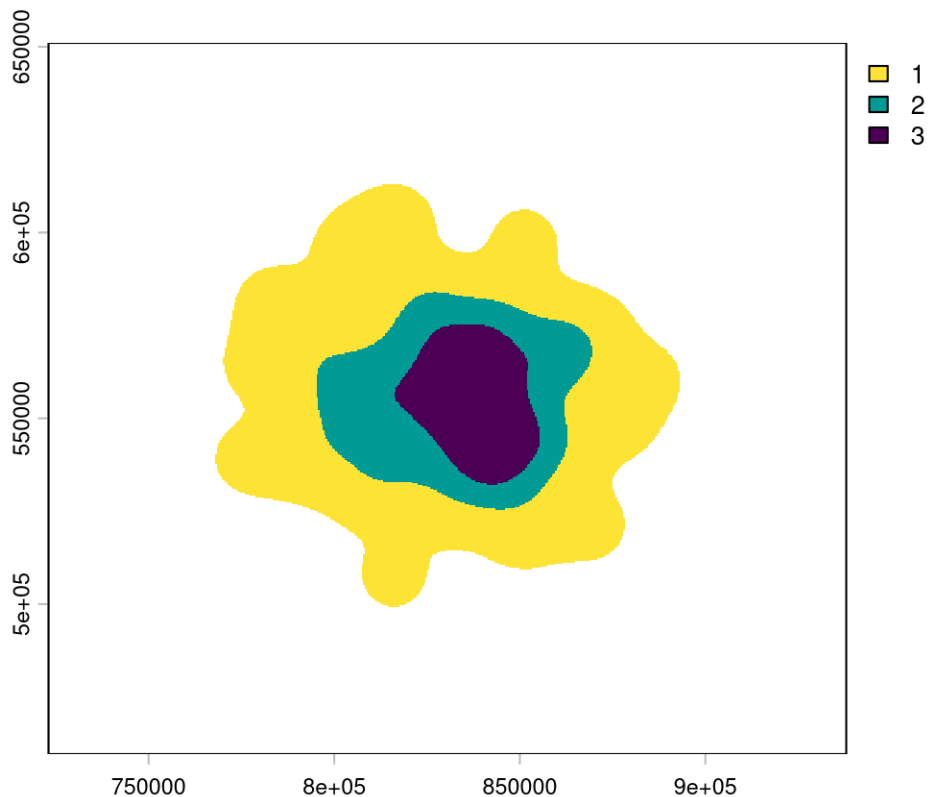
# Plot it
terra::plot(seal_rast_rec, col=hcl.colors(3, rev=TRUE))
```



The *Seal* data are ready to use. Now, we need to focus on the *Sea lion* data stored in `sea_lion_df`. We will create the map using the `get_class_kernel()` function from **risa**.

```
# KDE
sea_lion_kde <- get_class_kernel(sea_lion_df)

# Plot
terra::plot(sea_lion_kde, col=hcl.colors(3, rev=TRUE))
```



With this, the species data are ready. Now we need to work on the stressor data. Both `gillnet_sf` and `driftnet_sf` are `sf` objects containing multiple polygons that represent fishing areas reported by different fishermen. Our goal is to identify areas with greater overlap among these polygons, which can be interpreted as zones of higher fishing activity.

To do this, we first add a `Rating` attribute to each polygon and set its value to 1. Then, we rasterize the polygons and sum their `Rating` values. As a result, raster cells covered by one polygon receive value 1, cells where two polygons overlap receive value 2, and so on. This creates a raster map representing the intensity of overlapping fishing use.

```
# Convert sf to SpatVector
v_gillnet <- terra::vect(gillnet_sf)
v_driftnet <- terra::vect(driftnet_sf)

# Add value 1 of 'Rating' to each polygon
v_gillnet$Rating <- 1
v_driftnet$Rating <- 1

# Reproject to a metric CRS before defining resolution in meters
# Example: WGS 84 / UTM zone 30N (EPSG:32630)
v_gillnet_m <- terra::project(v_gillnet, "EPSG:32630")
v_driftnet_m <- terra::project(v_driftnet, "EPSG:32630")

# Create a template raster (adjust res according to scale!)
template_gillnet <- terra::rast(
  terra::ext(v_gillnet_m),
  res = 500,           # pixel size in meters
```

```

crs = terra::crs(v_gillnet_m))

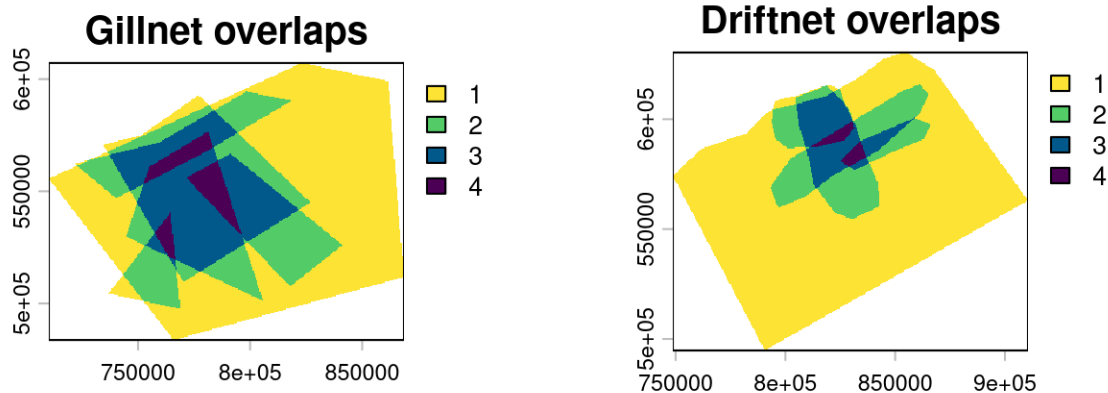
template_driftnet <- terra::rast(
  terra::ext(v_driftnet_m),
  res = 500,
  crs = terra::crs(v_driftnet_m))

# Rasterize and sum overlapping polygons
gillnet_overlap_r <- terra::rasterize(
  v_gillnet_m,
  template_gillnet,
  field = "Rating",
  fun = "sum",
  background = NA)

driftnet_overlap_r <- terra::rasterize(
  v_driftnet_m,
  template_driftnet,
  field = "Rating",
  fun = "sum",
  background = NA)

# Plot result
par(mfrow = c(1,2))
terra::plot(gillnet_overlap_r,
  main = "Gillnet overlaps",
  col=hcl.colors(5, rev=TRUE))
terra::plot(driftnet_overlap_r,
  main = "Driftnet overlaps",
  col=hcl.colors(5, rev=TRUE))

```

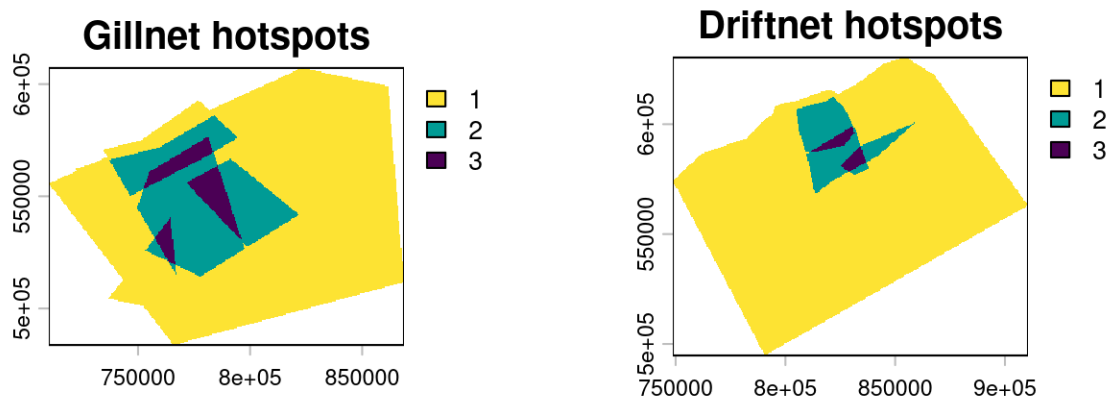


The next step is to reclassify these fishery overlap maps into three ordinal categories.

```
# Build reclassification matrices for each stressor
gillnet_re_mat <- reclass_matrix(gillnet_overlap_r,
                                exclude_lowest = FALSE)
driftnet_re_mat <- reclass_matrix(driftnet_overlap_r,
                                  exclude_lowest = FALSE)

# Reclassify maps
gillnet_hotspots <- terra::classify(gillnet_overlap_r,
                                    gillnet_re_mat,
                                    include_lowest = TRUE)
driftnet_hotspots <- terra::classify(driftnet_overlap_r,
                                     driftnet_re_mat,
                                     include_lowest = TRUE)

# Plot reclassified maps
par(mfrow = c(1,2))
terra::plot(gillnet_hotspots,
            main = "Gillnet hotspots",
            col=hcl.colors(3, rev=TRUE))
terra::plot(driftnet_hotspots,
            main = "Driftnet hotspots",
            col=hcl.colors(3, rev=TRUE))
```



Nice! Now we have our maps almost ready to go. We still have to harmonize their coordinates, extensions, and resolution. Luckily, **risa** has a function for that called **byra_prep()**. All we need to do is build two named lists, one for species and the other for stressors. We have to pay attention on how we name each map, since the names must match the names in the criteria table. The names in the criteria table for species are **Sea.lion** and **Seal**, and for stressors are **Gillnet** and **Driftnet**.

```
# Preparing input lists
species_list <- list(Sea.lion = sea_lion_kde,
                    Seal = seal_rast_rec)

stressor_list <- list(Gillnet = gillnet_hotspots,
                    Driftnet = driftnet_hotspots)

# Harmonize maps for ByRA
byra_input <- byra_prep(species_list,
                       stressor_list,
                       quiet = FALSE)
```

```
## Area is NULL.
```

```
## Input maps do not have the same coordinate system.
```

```
## All rasters match the template geometry (extent/res/origin).
```

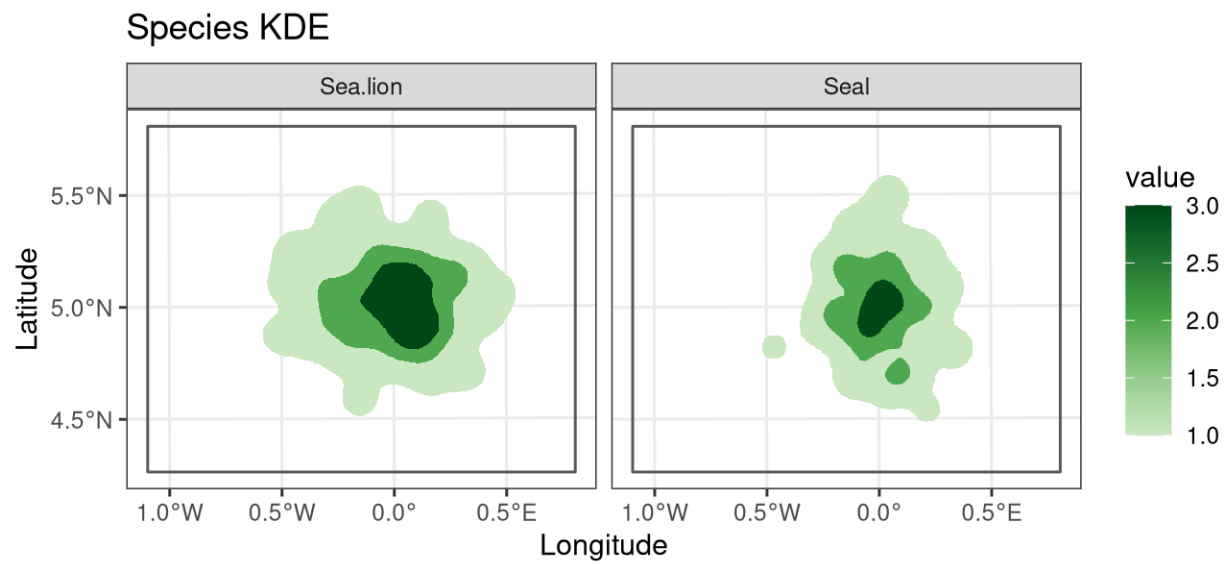
```
## No area provided. Creating AOI (bounding box) from all raster layers.
```

The **byra_prep()** function not only harmonizes all input maps to a common template with the same CRS, extent, and resolution, but also computes additional input maps required by the HRA model, such as the reclassified spatial overlaps between species and stressor maps.

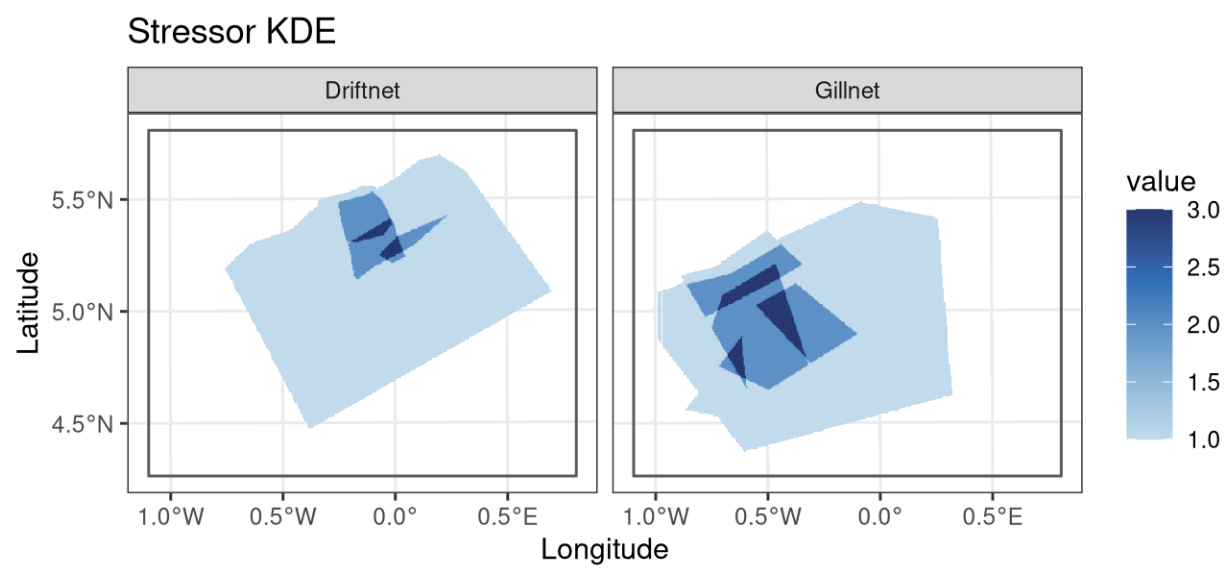
The output of `byra_prep()` is a `risaMaps` object, a list-like object used by several functions in **risa**, including `risaplot()`.

```
risaplot(byra_input)
```

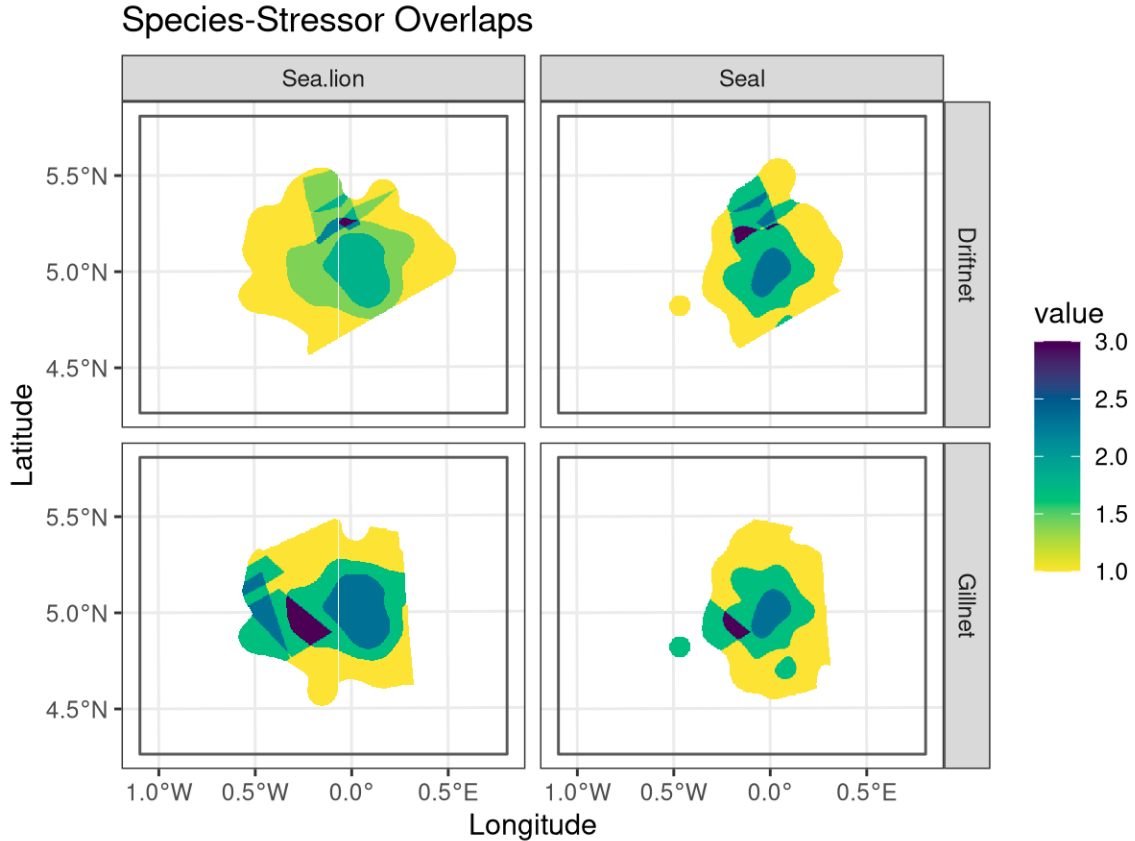
```
## [[1]]
```



```
##  
## [[2]]
```



```
##  
## [[3]]
```



Note that `risaplot()` automatically assigns standard titles to the plots, such as "Species KDE" and "Stressor KDE". In this example, however, only one panel, the *Sea lion* panel, is actually a KDE map. You can ignore these titles, as they are simply part of the default output produced by `risaplot()`.

We now have all the input maps ready to run the model. Let's run it again using `quick_byra()`. This time, we will use the *Euclidean* equation for the model estimates.

```
byra3 <- quick_byra(byra_input,
  criteria = criteria2,
  equation = "euclidean",
  quiet = FALSE)
```

Input x is a `risaMaps` object.

Risk equation: `euclidean`

Max Rating score: 3

Number of overlapping stressors: 2

And that is it! Let's check out the output! First the stats.

```
byra3$summary_stats
```

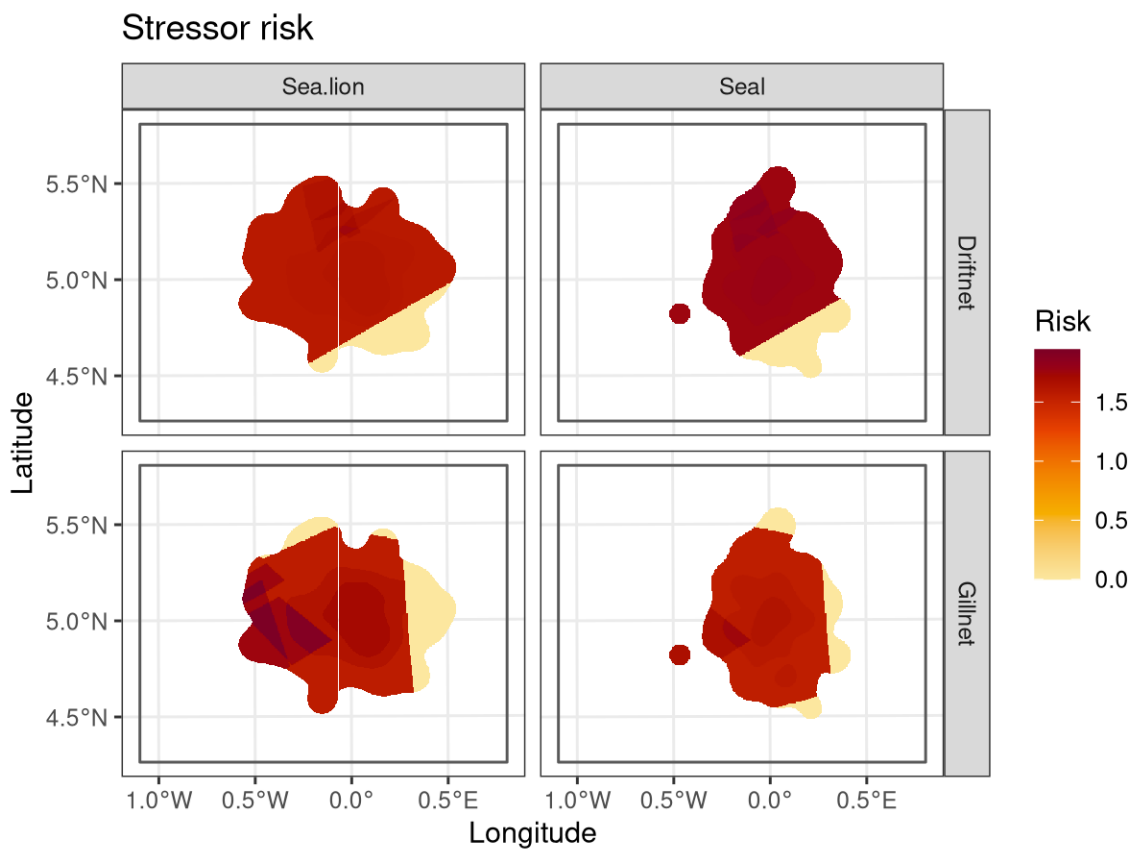
##	SPECIES	STRESSOR	E_min	E_max	E_mean	C_min	C_max
## 3	ECOSYSTEM (FROM ALL STRESSORS)		NA	NA	NA	NA	NA
## 1	Sea.lion (FROM ALL STRESSORS)		0	2.333333	1.727487	0	2.512821
## 2	Sea.lion	Gillnet	0	2.333333	1.736625	0	2.512821
## 31	Sea.lion	Driftnet	0	2.068966	1.718349	0	2.385542
## 4	Seal (FROM ALL STRESSORS)		0	2.181818	1.588874	0	2.700000

```
## 5      Seal      Gillnet      0 2.181818 1.761683      0 2.285714
## 6      Seal      Driftnet      0 1.800000 1.416064      0 2.700000
##      C_mean R_min      R_max      R_mean      R.high R.medium      R.low      R.None
## 3      NA      0 3.535054 2.718769 0.000000 70.07286 24.69617 5.230971
## 1 1.942399      0 1.944849 1.396128 0.000000 74.55457 21.89215 3.553282
## 2 1.861553      0 1.944849 1.376387 7.509654 75.35567 0.00000 17.134679
## 31 2.023244      0 1.749976 1.415868 0.000000 88.13597 0.00000 11.864034
## 4 2.041849      0 1.862927 1.421406 0.000000 74.82428 19.91453 5.261191
## 5 1.906739      0 1.746355 1.372214 0.000000 86.68006 0.00000 13.319945
## 6 2.176959      0 1.862927 1.470599 0.000000 82.88303 0.00000 17.116970
```

And now the mapped risk estimates:

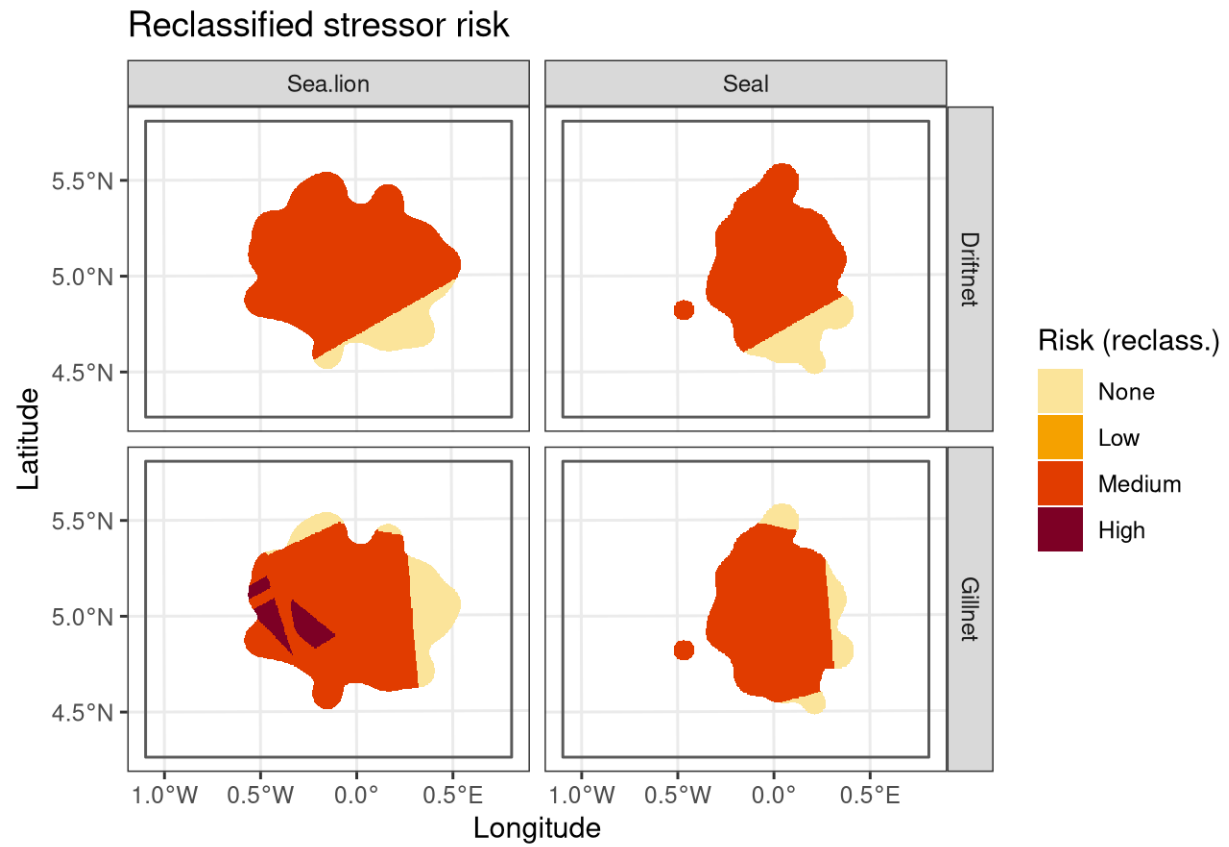
```
risaplot(byra3)
```

```
## [[1]]
```

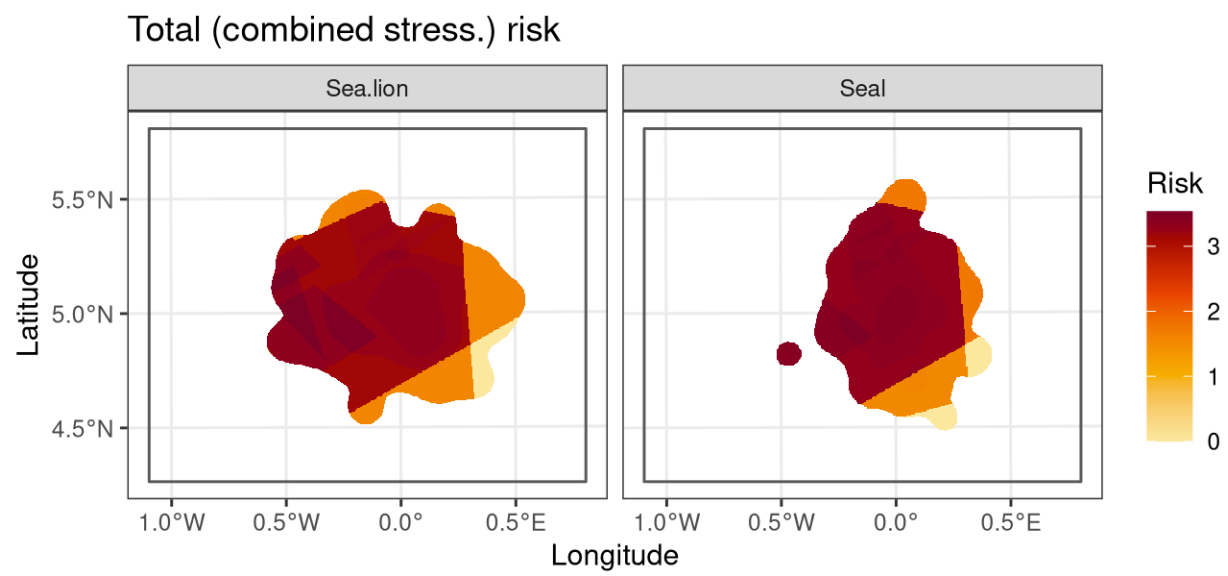


```
##
```

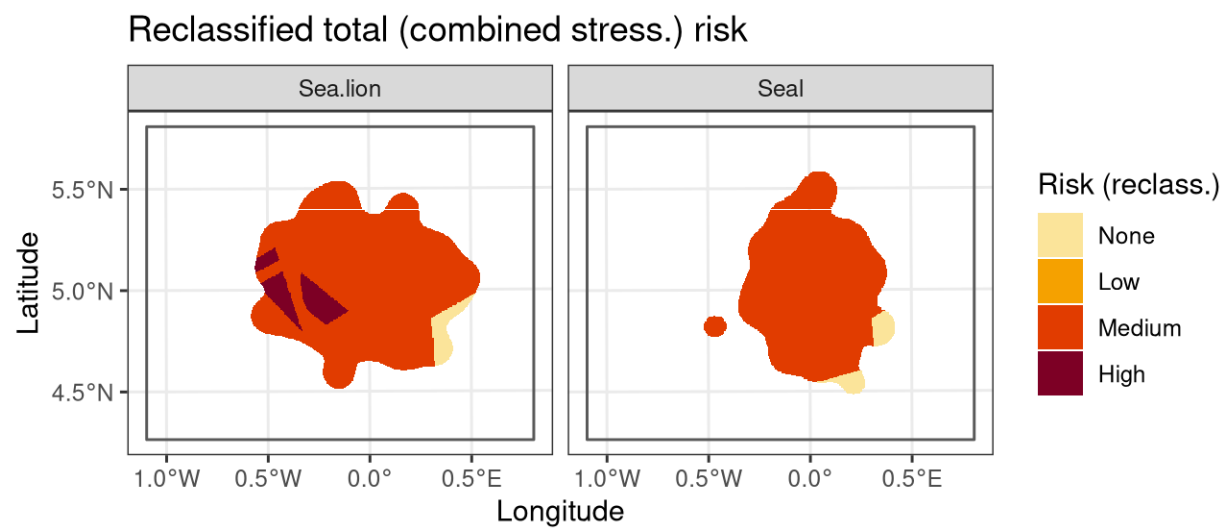
```
## [[2]]
```



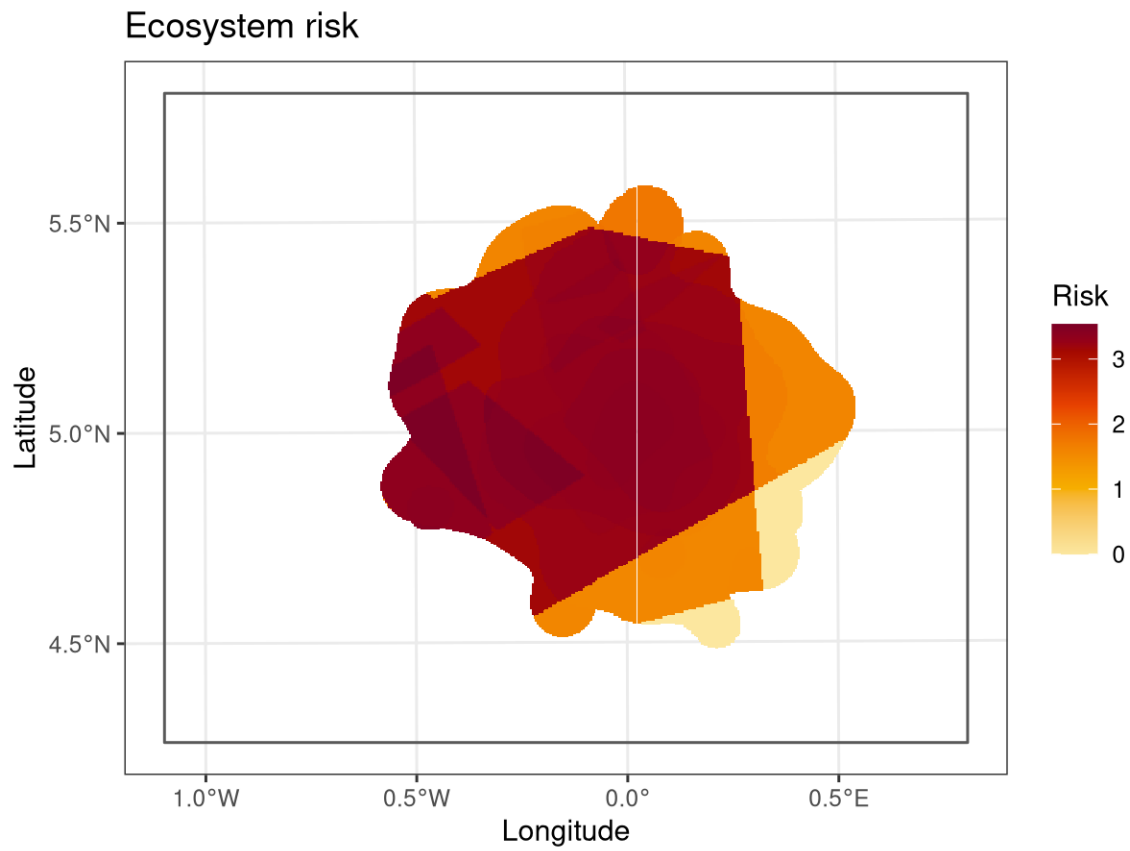
[[3]]



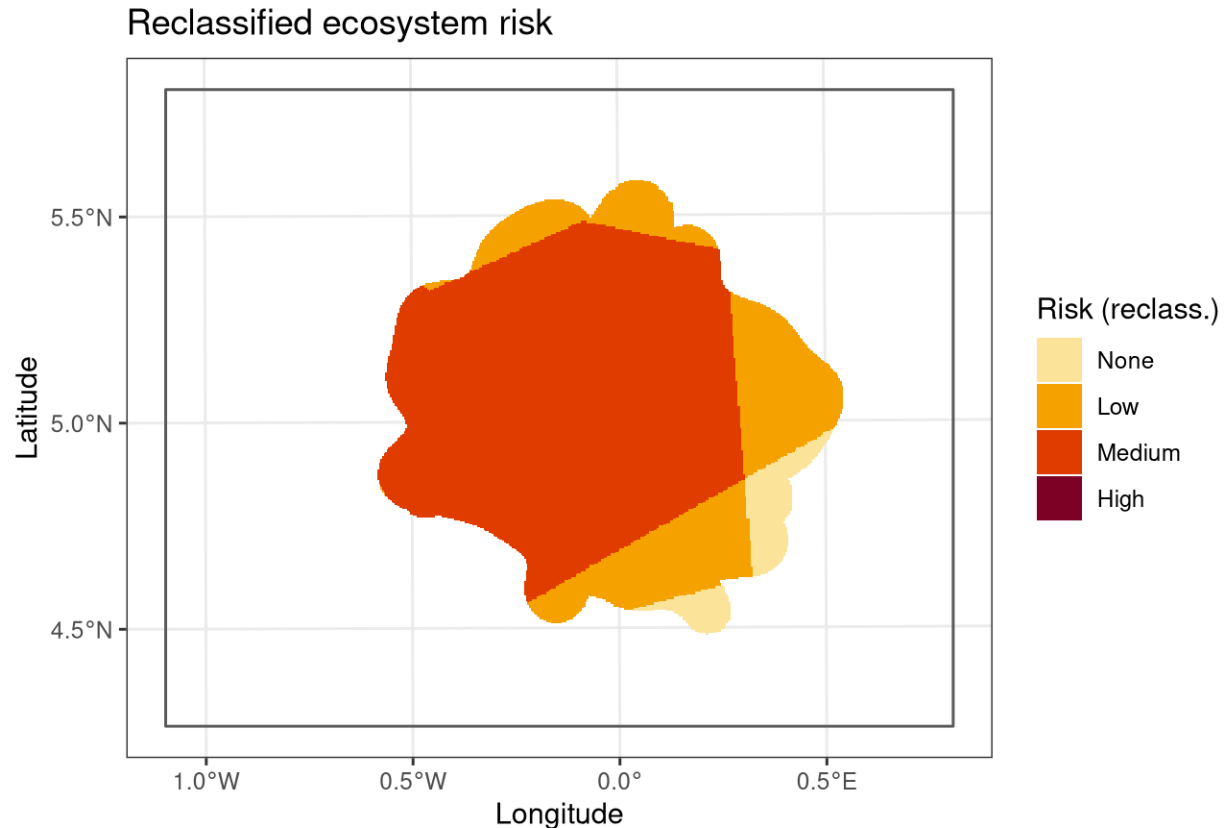
[[4]]



[[5]]



```
##  
## [[6]]
```



Further applications of the HRA model

For more complex HRA models that differ from the standard structure typically used in *ByRA*, we recommend using the `hra()` function in **risa**. Since this tutorial focuses mainly on applying *ByRA* in *R*, we will include only a brief example showing how to run the HRA model directly with `hra()` using the same example data (`spp_df`, `str_df`, and `criteria`).

First, we need to create the input maps from our data. This can be done with `risa_prep()`, which takes the species and stressor data frames (`spp_df` and `str_df`) as inputs. If we use the same values for `area_strategy`, `radius`, and `group_size_x` that were used in `byra2`, the output from `risa_prep()` will be equivalent to the objects stored in `byra2_kdes`.

```
input_kdes <- risa_prep(spp_df, str_df,
  area_strategy = "union",
  radius = c(Dolphin = 12000,
    Turtle = 7000,
    Gillnet = 10000,
    Trawling = 8000),
  group_size_x = "group_size")
```

We now have the input maps, but they must be organized into the correct structure before running `hra()`.

The input maps must be arranged as a hierarchical list of lists. At the top level, the object should contain one list for each species. Within each species-level list, there should be one list for each stressor. Finally, within each stressor-level list, the maps should be stored as rasters named according to the corresponding attributes in the criteria table.

In the case of *ByRA*, these attributes are "intensity" and "likelihood of interaction". However, in

other HRA-based frameworks, the mapped attributes may differ depending on the risk assessment context. For example, they may describe attributes associated with stressors other than fishing gear types, or they may be defined for habitat types rather than for individual species.

In this example, the input list (`rast_list`) should have the following structure:

```
rast_list
|-- Dolphin
|   |-- Gillnet
|   |   |-- intensity
|   |   |-- likelihood of interaction
|   |-- Trawling
|       |-- intensity
|       |-- likelihood of interaction
`-- Turtle
    |-- Gillnet
    |   |-- intensity
    |   |-- likelihood of interaction
    |-- Trawling
        |-- intensity
        |-- likelihood of interaction
```

We also need a list of maps, `spp_dist`, describing the overall distribution of each species. These maps are derived from the species KDE maps but are reclassified to represent areas of potential presence as 1 and areas of absence as NA.

Let's build the input lists in R:

```
rast_list <- list(
  Dolphin = list(
    Gillnet = list(
      intensity = input_kdes$stressor_kernel_maps$Gillnet$raster,
      `likelihood of interaction` = input_kdes$overlap_maps$Dolphin$Gillnet$raster),
    Trawling = list(
      intensity = input_kdes$stressor_kernel_maps$Trawling$raster,
      `likelihood of interaction` = input_kdes$overlap_maps$Dolphin$Trawling$raster)
  ),
  Turtle = list(
    Gillnet = list(
      intensity = input_kdes$stressor_kernel_maps$Gillnet$raster,
      `likelihood of interaction` = input_kdes$overlap_maps$Turtle$Gillnet$raster),
    Trawling = list(
      intensity = input_kdes$stressor_kernel_maps$Trawling$raster,
      `likelihood of interaction` = input_kdes$overlap_maps$Turtle$Trawling$raster)
  )
)

# Prepare lists with Species' distribution maps
spp_dist <- list(Dolphin = input_kdes$species_distributions$Dolphin$raster,
                 Turtle = input_kdes$species_distributions$Turtle$raster)

# For the hra(), we need to reshape the criteria table
crit_list <- criteria_reshape(criteria)
```

Now we have everything needed to run the model. Most arguments in `hra()` are similar to those in `quick_byra()`, because `quick_byra()` uses `hra()` internally to perform the analysis.

However, `hra()` gives users more control over the structure of the input objects. This makes it useful for implementing extensions of the HRA model, including models based on different criteria attributes and associated input maps.

We can also account for the spatial zone of influence around stressor observations. For example, suppose that **Gillnet** has an influence zone of 10000 m, while **Trawling** has an influence zone of 7000 m. We can represent this by using a linear decay function with `decay = "linear"` and defining stressor-specific buffers with:

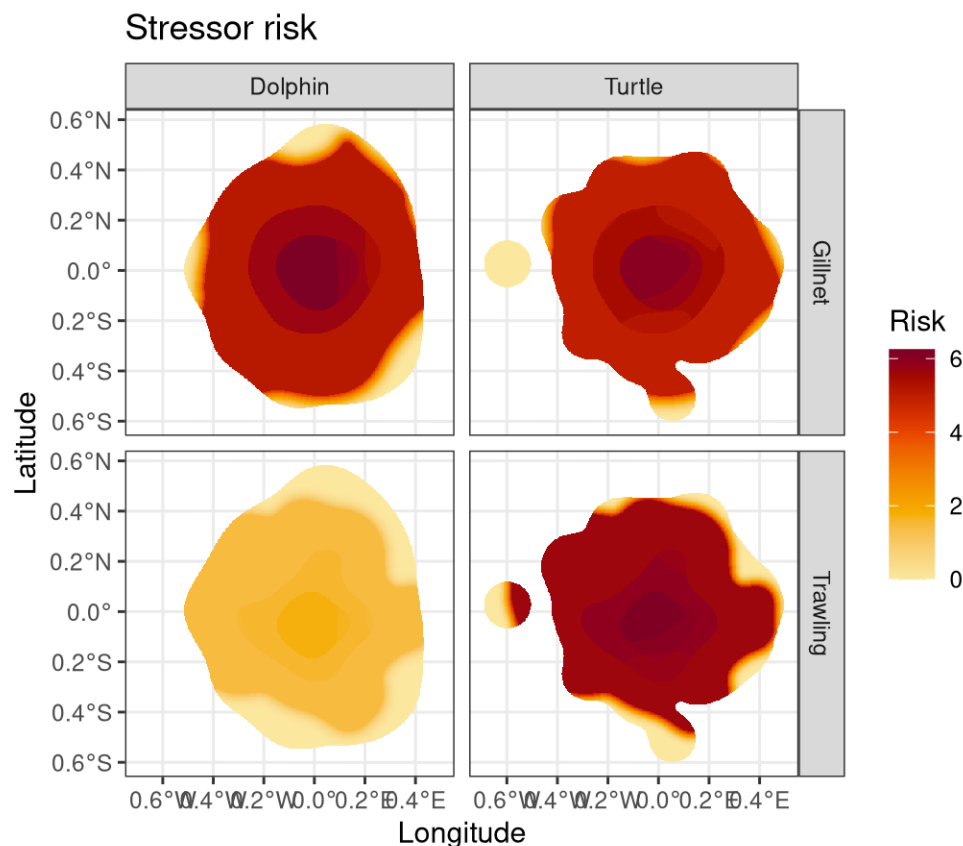
```
buffer_m = c(Gillnet = 10000, Trawling = 7000)
```

Because a decay function is applied to the stressors, we should expect to see gradients of risk around the core stressor KDEs, especially in areas where stressors overlap with species distributions.

```
# Perform the HRA model
hra1 <- hra(raster_list = rast_list,
           species_distr = spp_dist,
           criteria = crit_list,
           equation = "multiplicative",
           decay = "linear",
           buffer_m = c(Gillnet = 10000, Trawling = 7000))
```

The output of `hra()` follows the same structure as the output from `quick_byra()` and can also be plotted using `risaplot()`. Here, we will inspect only the first plot output to visualize the buffers generated by the decay function around the stressor distributions for each species.

```
hra_plots <- risaplot(hra1)
hra_plots[[1]]
```



Exporting maps

Now that the results are organized in nested lists, it is useful to have a simple way to export them as *GeoTIFF* or *shapefile* files, so they can be opened in QGIS or in any other geographic information system. For this purpose, **risa** includes the `export_maps()` function, which provides a quick way to export outputs generated by functions such as `quick_byra()`, `hra()`, `byra_prep()`, and `risa_prep()`.

The `export_maps()` function exports the full output object into a nested folder structure that mirrors the list-of-lists structure of the results. In this example, we will export the output of our first *ByRA* model, stored in the `byra1` object.

```
# It will create a new folder inside your working directory
export_maps(byra1, "output_folder_name")
```

Making pretty maps

I know that some of you probably want to make your maps in R. So, we prepared this section so you can make figures that are good enough to be published in a paper (or at least good enough for a report). So, let's do it!

First, since we want to make maps, let's ask `quick_byra()` to give us maps in decimal CRS (WGS 84, EPSG:4326), which is much easier to plot with other ready-to-use layers we will grab from useful packages such as **geodata**.

```
# This time, we ask for decimal degrees by setting return_crs = "4326"
byra4 <- quick_byra(byra_input,
                    criteria = criteria2,
                    equation = "euclidean",
                    return_crs = "4326",
                    quiet = FALSE)
```

```
## Input x is a risaMaps object.
```

```
## Risk equation: euclidean
```

```
## Max Rating score: 3
```

```
## Number of overlapping stressors: 2
```

To make our plots pretty, we need to load a couple of packages. The first is **geodata**, which provides vectorial maps of different parts of the worlds. The second is **ggspatial**, that provides functions to complement our ggplot2 maps with scale bars and north arrows. If you don't have these, you must install them using `install.packages(c("geodata", "ggspatial"))`.

```
# Loading the packages (install if you don't have them)
library(geodata)
library(ggspatial)
```

The next step would be to load the land layers or any spatial features we want to represent together with the bycatch risk maps we made. We will download a vectorial world map using the `geodata::world()` function and then extract the map of Ghana for our map.

```
# Load world map data
world_map <- geodata::world(resolution = 3, path = "data/")

# We don't need all the countries, so we extract only what we need
ghana <- world_map[world_map$NAME_0 %in% c("Ghana"), ]

# Let's also include some administrative boundaries, like municipalities
ghana_adm <- geodata::gadm(country = "GHA", level = 2, path = tempdir()) # level varies by country
```

```
## Cached as: /tmp/RtmpzFVPkG/gadm/gadm41_GHA_2_pk.rds
```

```
# Finally, we need to convert these vectors to the 'sf' class
```

```
ghana_sf <- sf::st_as_sf(ghana)
```

```
ghana_adm_sf <- sf::st_as_sf(ghana_adm)
```

We can make quick and acceptable maps using the standard output from `risaplot()`...

```
# Making ggplot-type risk maps with risaplot
```

```
risk_plots <- risaplot(byra4)
```

```
# Let's plot the map of the reclassified ecosystem risk
```

```
risk_plots[[6]] +
```

```
  geom_sf(data=ghana_sf, fill="gray", col="black") + # include the land polygon
```

```
  geom_sf(data=ghana_adm_sf, fill="#00000000", col="gray50") + # include the municipalities
```

```
  coord_sf(xlim=c(-1.15,1), # defines the zoom window
```

```
            ylim=c(4,6)) +
```

```
  annotation_scale(location = "br", # include scale bar
```

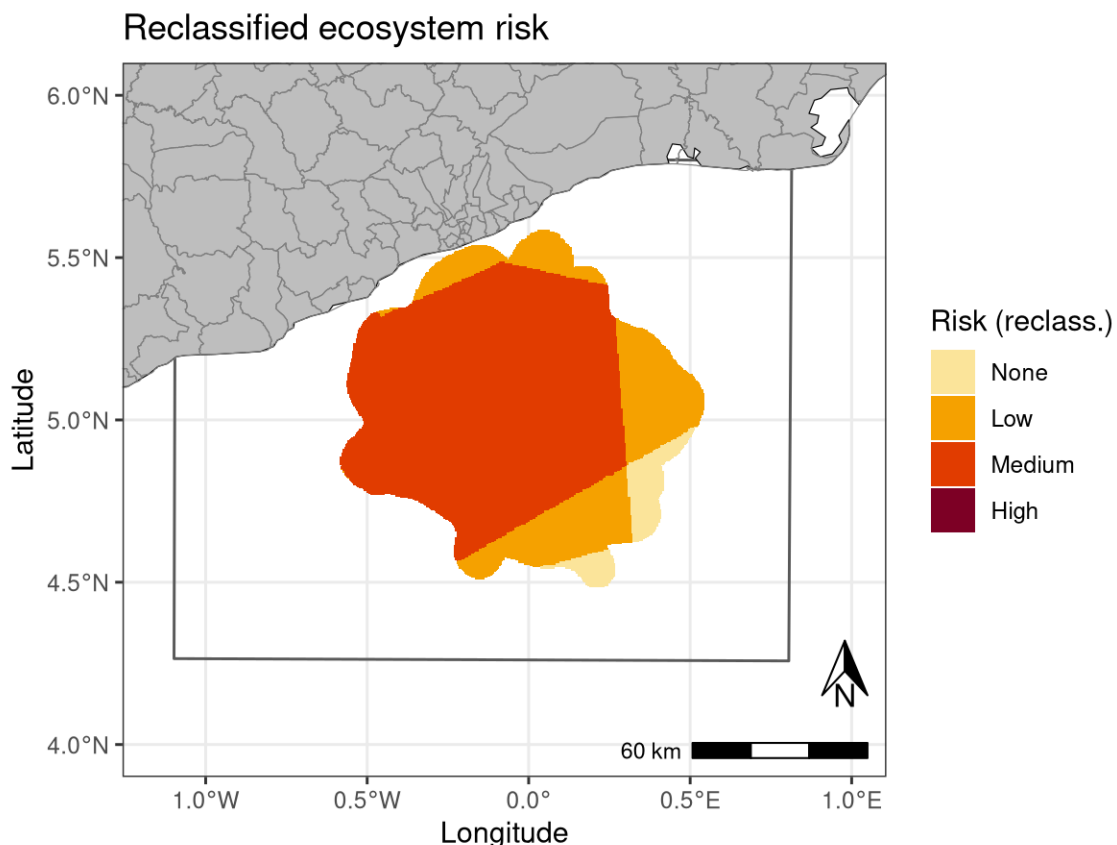
```
                    height = unit(0.2, "cm")) +
```

```
  annotation_north_arrow(location = "br", # include North arrow
```

```
                         pad_y=unit(1, "cm"),
```

```
                         width = unit(0.6, "cm"),
```

```
                         height = unit(0.8, "cm"))
```



... or we can generate “raw” ggplot maps by adding the `raw_ggplot = TRUE` and further edit it as we wish...

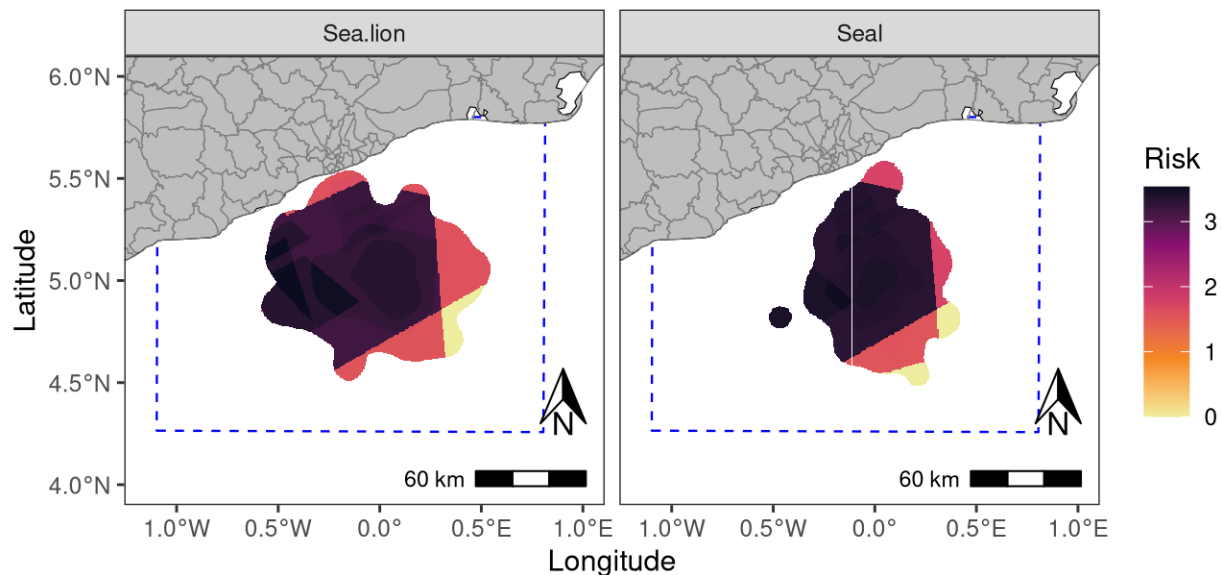
```

# Making raw-ggplot maps without the area of interest
clean_kde_plots <- risaplot(byra4, raw_ggplot = TRUE, area_of_interest = FALSE)

# Defining a new color palette
palette <- c("#EEEC9E", "#F58924", "#D64267", "#891171", "#0C0C21")

# Let's use
clean_kde_plots[[3]] +
  # This time, I will set a new color palette
  scale_fill_gradientn(colors=palette) +
  # And now including area of interest from our byra4 output, but with some edits
  geom_sf(data=byra4$area_of_interest, fill="#00000000", col="#0000FF",
    size=0.4, linetype = "dashed") +
  geom_sf(data=ghana_sf, fill="gray", col="black") +
  geom_sf(data=ghana_adm_sf, fill="#00000000", col="gray50") +
  coord_sf(xlim=c(-1.15,1),
    ylim=c(4,6)) +
  annotation_scale(location = "br",
    height = unit(0.2, "cm")) +
  annotation_north_arrow(location = "br",
    pad_y=unit(1, "cm"),
    width = unit(0.6, "cm"),
    height = unit(0.8, "cm")) +
  theme_bw() + # Making theme black and white
  theme(panel.grid = element_blank()) # removing gridlines

```



... or we can use the layers from our byra4 output and make our maps from zero. The `quick_byra()` outputs are in *SpatRaster* format, so we need to convert them into *data.frame* in order to plot them using **ggplot2**. Also, since we will combine multiple ggplots into a single figure, we will use the **patchwork** package (install if you don't have it!).

Let's suppose we want to map the risk estimates for sea lions and the combined risk estimates for all pennipeds.

```
# Load patchwork package
library(patchwork)

# First, let's extract what we need
sealion_total <- byra4$Sea.lion$total_raw
ecosys_total <- byra4$ecosys_risk_raw
aio_sf <- byra4$area_of_interest

# The ecosystem risk raster needs a small adjustment in its value name
names(ecosys_total) <- "Rating"

# Since risk maps are SpatRaster objects, so we need to convert them to data.frame
sealion_total_df <- as.data.frame(sealion_total, xy=TRUE, na.rm=TRUE)
ecosys_total_df <- as.data.frame(ecosys_total, xy=TRUE, na.rm=TRUE)

# Check out how the data looks like!
head(sealion_total_df)

##           x           y Rating
## 52802 -0.1752729 5.536787 1.590204
## 52803 -0.1716876 5.536787 1.590204
## 52804 -0.1681022 5.536787 1.590204
## 52805 -0.1645169 5.536787 1.590204
## 52806 -0.1609316 5.536787 1.590204
## 52807 -0.1573462 5.536787 1.590204
```

Neat! Now let's define the aesthetics of our plots, strating from the color palette and them the plot stucture.

```
# This time, we will define a different palete for species and ecosystem maps
spp_palette <- c("#DCE95B", "#32AE7C", "#255668")
eco_palette <- c("#FFD99F", "#F7606D", "#7D1D67")

# Since we will standardize the aesthetics of the plots, we can define a template
template <- ggplot() +
  geom_sf(data=byra4$area_of_interest, fill="#00000000", col="gray70",
    size=0.4, linetype = "dashed") +
  geom_sf(data=ghana_sf, fill="gray", col="black") +
  geom_sf(data=ghana_adm_sf, fill="#00000000", col="gray50") +
  coord_sf(xlim=c(-1.15,1), # Defines zoom in window
    ylim=c(4,6)) +
  theme_bw() +
  theme(panel.grid = element_blank()) +
  labs(x = "Longitude", y="Latitude") # Defines x and y axes labels
```

Ok! Let's make the ggplots for sea lions and all pennipeds...

```
# Sea lion
sp1 <- template +
  geom_tile(data=sealion_total_df, aes(x=x, y=y, fill=Rating)) +
  scale_fill_gradientn(colors=spp_palette) +
```

```

ggtitle("a) Sea lion (total risk)") # Includes a title

# All pennipeds combined
tot <- template +
  geom_tile(data=ecosys_total_df, aes(x=x, y=y, fill=Rating)) +
  scale_fill_gradientn(colors=eco_palette) +
  ggtitle("b) Pennipeds (combined total risk)") +
  # Let's add a scale bar and a north arrow to this one
  annotation_north_arrow(location = "br", # North arrow
                        pad_y=unit(1, "cm"),
                        width = unit(0.6, "cm"),
                        height = unit(0.8, "cm")) +
  annotation_scale(location = "br", # Scale bar
                  height = unit(0.2, "cm"))

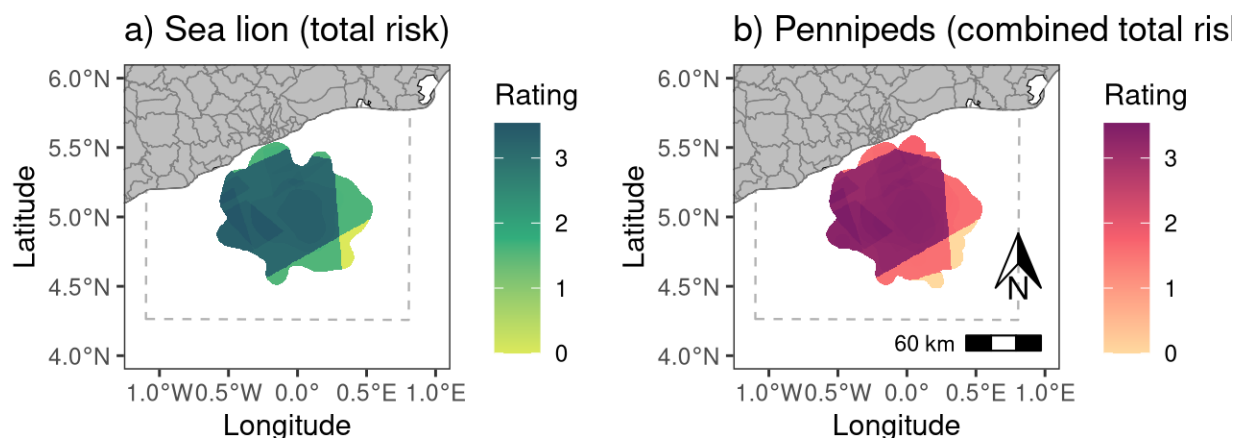
```

... and plot them using the “|” **patchwork** operator.

```

# Plot them side-by-side
sp1 | tot

```



Concluding remarks

This tutorial introduced the main functions available in the **risa** package to perform *ByRA*, as well as other possible extensions of the HRA model.

In addition to risk assessment workflows, **risa** also includes tools for data exploration and rapid assessment of species and stressor distribution hotspots, spatial overlap patterns, and input map preparation.

We hope you find the package useful. Please note that **risa** is currently in an *alpha* version, so testing with real datasets is especially valuable at this stage. Criticism, suggestions, feature requests, and bug reports are very welcome.

You can contribute or report issues through the **risa** GitHub repository:

<https://github.com/GusmaoJB/risa/>

You can also contact me directly by email:

gusmao.jb@gmail.com

References

- Arkema, K. K., Verutes, G., Bernhardt, J. R., Clarke, C., Rosado, S., Canto, M., Wood, S. A., Ruckelshaus, M., Rosenthal, A., McField, M., & De Zegher, J. (2014). Assessing habitat risk from human activities to inform coastal and marine spatial planning: A demonstration in Belize. *Environmental Research Letters*, 9, 114016. <https://doi.org/10.1088/1748-9326/9/11/114016>
- Hines, E. M., Johnson, A., Ponnampalam, L., Peter, C., Junchompoo, C., Vu, L., Thien, H., Caillat, M., & Verutes, G. (2020). Getting to the bottom of bycatch: A GIS-based toolbox to assess the risk of marine mammal bycatch. *Endangered Species Research*, 42, 37–57. <https://doi.org/10.3354/esr01037>
- Sharp, R., Douglass, J., Wolny, S., Arkema, K., Bernhardt, J., Bierbower, W., Chaumont, N., Denu, D., Fisher, D., Glowinski, K., Griffin, R., Guannel, G., Guerry, A., Johnson, J., Hamel, P., Kennedy, C., Kim, C. K., Lacayo, M., Lonsdorf, E., Mandle, L., Rogers, L., Silver, J., Toft, J., Verutes, G., Vogl, A. L., Wood, S., & Wyatt, K. (2020). *InVEST 3.9.0 user's guide*. The Natural Capital Project, Stanford University, University of Minnesota, The Nature Conservancy, and World Wildlife Fund.
- Verutes, G. M., Johnson, A. F., Caillat, M., Ponnampalam, L. S., Peter, C., Vu, L., & Hines, E. M. (2020). Using GIS and stakeholder involvement to innovate marine mammal bycatch risk assessment in data-limited fisheries. *PLOS ONE*, 15(8), e0237835. <https://doi.org/10.1371/journal.pone.0237835>